

TEXTBOOK EDITION • RELATIONAL DATABASES & CONCURRENCY

SQL Mastery, Concurrency & Advanced DBMS

Banking Relational Modeling, Indexing, ACID, MVCC, Locks & Optimization

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	16 Hours
DIFFICULTY LEVEL	Banking Critical
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — SQL Fundamentals & Relational Data Modeling for Banking	Study Time: 35m
Chapter 02 — Advanced SQL Joins & Relational Join Algorithms	Study Time: 35m
Chapter 03 — SQL Subqueries, Correlated Execution & EXISTS vs IN	Study Time: 35m
Chapter 04 — SQL Window Functions for Financial Ledgers & Analytics	Study Time: 35m
Chapter 05 — Common Table Expressions (CTEs) & Recursive CTEs	Study Time: 35m
Chapter 06 — ACID Properties & Database Transaction Internals	Study Time: 35m
Chapter 07 — Database Isolation Levels, Read Phenomena & MVCC	Study Time: 35m
Chapter 08 — Database Locking, Pessimistic vs Optimistic Concurrency & Deadlocks	Study Time: 35m
Chapter 09 — Most Asked & Tricky SQL & DBMS Interview Questions	Study Time: 35m
Chapter 10 — SQL & DBMS Last-Minute Interview Cheatsheet	Study Time: 35m

CHAPTER 01

SQL Fundamentals & Relational Data Modeling for Banking

PRIORITY

 High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

SQL Fundamentals & Relational Data Modeling for Banking

1. Why This Matters

In banking systems like IDFC FIRST Bank, relational databases are the source of truth for accounts, balances, regulatory audit trails, and ledgers. Unlike web startups that can tolerate eventual consistency or lost metrics, a banking system cannot lose a single rupee or permit duplicate debits. 3+ YoE interviewers test whether you truly understand relational integrity constraints, DDL/DML semantics, schema design trade-offs, and query execution fundamentals.

2. Prerequisites

- Basic understanding of relational tables (rows, columns, schemas).
- Familiarity with SQL syntax (`SELECT` , `INSERT` , `UPDATE` , `DELETE`).

3. Concept

Relational Integrity Constraints

- Primary Key (PK):** Uniquely identifies each record; implies `NOT NULL` and `UNIQUE` . In transactional banking, auto-incrementing integers or sequential UUIDv7 are preferred over random UUIDv4 for clustered index insertion efficiency.
- Foreign Key (FK):** Enforces referential integrity between child and parent tables (`ON DELETE RESTRICT` prevents orphaned accounts if a customer record is deleted).
- Check Constraints (`CHECK`):** Enforces domain validation at the database layer (e.g., `CHECK (balance ≥ 0.00)` prevents overdraft below zero at the storage engine level, regardless of application bugs).
- Unique Constraints (`UNIQUE`):** Enforces non-duplication (e.g., PAN numbers, account numbers, idempotency keys).

```

erDiagram
    CUSTOMERS ||--o{ ACCOUNTS : owns
    ACCOUNTS ||--o{ TRANSACTIONS : records
    ACCOUNTS ||--o{ CARDS : links
    CUSTOMERS ||--o{ BENEFICIARIES : manages
    CUSTOMERS ||--o{ LOANS : borrows
    CUSTOMERS ||--o{ PAYMENTS : initiates

    CUSTOMERS {
        int customer_id PK
        string pan_number UK
        string email UK
        string kyc_status
    }
    ACCOUNTS {
        int account_id PK
        int customer_id FK
        string account_number UK
        decimal balance "CHECK ≥ 0"
        int version "Optimistic Locking"
    }
    TRANSACTIONS {
        int transaction_id PK
        string transaction_reference UK
        int from_account_id FK
        int to_account_id FK
        decimal amount "CHECK > 0"
        string status
    }

```

4. Simple Example: Creating and Querying Bank Accounts

```

-- Creating an account with strict constraints
CREATE TABLE accounts_demo (
    account_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    account_type VARCHAR(20) CHECK (account_type IN ('SAVINGS', 'CURRENT')),
    balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (balance ≥ 0.00),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Safe balance increment
UPDATE accounts_demo
SET balance = balance + 500.00
WHERE account_id = 101;

```

5. Real-World Banking Example: Atomic Fund Transfer Invariant

When ₹1,000 is transferred from Account A to Account B: 1. Account A balance must be ≥ 1000 (enforced by application check + DB constraint `CHECK (balance ≥ 0.00)`). 2. The debit and credit must occur in a single atomic transaction. 3. If Account A tries to go negative, the database aborts with a constraint violation error, rolling back the debit and ensuring money cannot be created or destroyed.

6. Code: Essential DQL Queries on `banking.db`

```
-- 1. High-Value Active Savings Accounts with Customer Details
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS full_name,
    c.phone,
    a.account_number,
    a.balance
FROM customers c
INNER JOIN accounts a ON c.customer_id = a.customer_id
WHERE a.account_type = 'SAVINGS'
    AND a.status = 'ACTIVE'
    AND a.balance ≥ 50000.00
ORDER BY a.balance DESC;

-- 2. Aggregating Total Inflow vs Outflow Across Channels
SELECT
    payment_channel,
    COUNT(*) AS total_transactions,
    SUM(CASE WHEN transaction_type = 'DEPOSIT' THEN amount ELSE 0 END) AS total_deposited,
    SUM(CASE WHEN transaction_type IN ('WITHDRAWAL', 'TRANSFER') THEN amount ELSE 0 END) AS
total_outflow
FROM transactions
WHERE status = 'SUCCESS'
GROUP BY payment_channel
ORDER BY total_transactions DESC;
```

7. How It Works Internally: The SQL Execution Order

When you write a SQL query, the database parses and executes clauses in a logical order that is very different from the written order:

```
flowchart TD
    S1["1. FROM & JOIN (Form the cross-product / cartesian base table)"] --> S2["2. ON (Apply join condition filters)"]
    S2 --> S3["3. WHERE (Filter individual rows before aggregation)"]
    S3 --> S4["4. GROUP BY (Aggregate rows into buckets)"]
    S4 --> S5["5. HAVING (Filter aggregated group rows)"]
    S5 --> S6["6. SELECT (Evaluate expressions, column projections)"]
    S6 --> S7["7. DISTINCT (Eliminate duplicates)"]
    S7 --> S8["8. ORDER BY (Sort results)"]
    S8 --> S9["9. LIMIT / OFFSET (Slice output window)"]
```

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Why this matters in interviews: You cannot use column aliases created in `SELECT` inside the `WHERE` clause because `WHERE` executes before `SELECT`.

8. Common Mistakes

1. Using `WHERE` Instead of `HAVING` for Aggregates: `WHERE SUM(amount) > 1000` throws a syntax error. Aggregated expressions must be filtered in `HAVING`.

2. **Float/Double for Currency:** Never use `FLOAT` or `DOUBLE` for monetary balances. Floating-point binary representation produces IEEE 754 precision errors (e.g., $0.1 + 0.2 = 0.30000000000000004$). Always use `DECIMAL(15, 2)` or `NUMERIC`, or store balances as integer paise/cents.
3. **Missing Foreign Key Indexes:** Relational engines do NOT automatically index foreign key columns in child tables. Without an index on `transactions(from_account_id)`, every join or cascade operation forces a full sequential table scan!

9. Performance / Complexity Matrix

Clause / Operation	Complexity Without Index	Complexity With B-Tree Index
Lookup by Primary Key	$O(\log N)$ (Clustered)	$O(\log N)$
Filter by Indexed Column (<code>WHERE status = 'ACTIVE'</code>)	$O(N)$ (Sequential Scan)	$O(\log N + K)$ (Index Range Scan)
Join on Foreign Key (<code>JOIN accounts ON ...</code>)	$O(N * M)$ (Nested Loop)	$O(N \log M)$ (Index Scan) or $O(N+M)$ (Hash Join)
GROUP BY without Index	$O(N \log N)$ (Sort-based) or $O(N)$ (Hash-based aggregation)	$O(N)$ (Index ordered traversal)
ORDER BY on Indexed Column	$O(N \log N)$ (Sort in TempDB/Memory)	$O(1)$ (Direct B-Tree index scan)

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is the difference between `WHERE` and `HAVING` clauses? Give an example where one works and the other fails.

Medium

- **Q:** Explain why storing monetary values as `FLOAT` is unacceptable in banking. What exact data type should be used in PostgreSQL/MySQL and why?

Hard

- **Q:** In a high-concurrency payment table with 500 million rows, an application runs `SELECT COUNT(*) FROM transactions WHERE customer_id = 101`. The query takes 4 seconds. How would you diagnose, explain, and optimize this query?

11. Follow-up Questions from Interviewer

- "If you add an index on `(customer_id, created_at)`, does the order of columns in the composite index matter for queries filtering by `created_at` alone?"
- "What is a Write-Ahead Log (WAL), and how does it ensure durability before data pages are written to disk?"

12. Model Answer: Diagnosing & Optimizing Query Execution

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "A query filtering transactions by `customer_id` is slow in production. Walk me through how you investigate and resolve it."

Model Answer: "First, I inspect the query execution plan using `EXPLAIN ANALYZE` to check whether the optimizer is performing a **Sequential Table Scan** or an **Index Scan**.

In a 500M row table, if `customer_id` is unindexed, the engine scans every single data page from disk, consuming massive I/O.

To resolve this: 1. I create a B+ Tree index on `transactions(customer_id)`. 2. If the query frequently fetches the most recent transactions for a customer (`ORDER BY created_at DESC LIMIT 10`), a single-column index still requires a post-fetch sort. I would replace it with a **composite index** on `(customer_id, created_at DESC)`. 3. By following the **leftmost prefix rule**, the B+ tree seeks directly to `customer_id = 101` and traverses the index in already-sorted chronological order, executing in sub-millisecond $O(\log N + K)$ time without any in-memory sort or full table scan."

13. Practical Exercise (Run on `banking.db`)

Open your terminal and run the following queries on `banking.db`:

```
sqlite3 banking.db
```

```
-- 1. Find all customers who have both a SAVINGS and a SALARY account:
SELECT c.customer_id, c.first_name, c.last_name
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
WHERE a.account_type IN ('SAVINGS', 'SALARY')
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(DISTINCT a.account_type) = 2;
```

```
-- 2. Find the top branch by total deposited balance:
SELECT branch_code, SUM(balance) AS total_branch_liquidity
FROM accounts
GROUP BY branch_code
ORDER BY total_branch_liquidity DESC
LIMIT 1;
```

14. Quick Revision

- Primary Key = `UNIQUE` + `NOT NULL` .
- Foreign Keys require explicit indexes in child tables for join performance.
- Logical Execution Order: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT` .
- Always store monetary amounts in `DECIMAL(p, s)` or integer smallest currency units (paise/cents).
- `CHECK (balance ≥ 0.00)` guards against race-condition overdrafts.

15. Interview Checklist

- ☐ Can recite the SQL logical execution pipeline from memory.
- ☐ Understands why `WHERE` cannot evaluate aggregate functions.
- ☐ Explains `FLOAT` precision pitfalls with binary IEEE 754 floating-point arithmetic.
- ☐ Understands sequential scan vs index range scan.
- ☐ Can write grouping, having, and conditional aggregation queries without syntax errors.



CHAPTER 01 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Basic understanding of relational tables (rows, columns, schemas).
- Familiarity with SQL syntax (``SELECT``, ``INSERT``, ``UPDATE``, ``DELETE``).
- Q: What is the difference between ``WHERE`` and ``HAVING`` clauses? Give an example where one works and the other fails.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

Advanced SQL Joins & Relational Join Algorithms

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Advanced SQL Joins & Relational Join Algorithms

1. Why This Matters

At IDFC FIRST Bank, database normalization splits customer PII, account ledgers, KYC statuses, and debit card records across separate tables to prevent update anomalies. Generating a customer's unified statement, detecting AML fraud networks, or routing funds requires multi-table joins. Senior technical interviewers will grill you on how database query planners choose between **Nested Loop Joins**, **Hash Joins**, and **Merge Joins**, and how missing indexes degrade join throughput.

2. Prerequisites

- Relational keys (Primary Key, Foreign Key).
- Basic SQL syntax (`SELECT ... FROM ... WHERE ...`).

3. Concept

Types of SQL Joins

- **INNER JOIN**: Returns records where the join predicate matches in *both* tables.
- **LEFT (OUTER) JOIN**: Returns *all* rows from the left table and matched rows from the right table. Unmatched right columns evaluate to `NULL`.
- **RIGHT (OUTER) JOIN**: Returns all rows from the right table and matched rows from the left.
- **FULL OUTER JOIN**: Returns rows when there is a match in either left or right table.
- **CROSS JOIN**: Produces the Cartesian product ($N * M$ rows).
- **SELF JOIN**: Joins a table to itself (used for hierarchical data, manager-employee structures, or transaction matching).

```
flowchart TD
    subgraph JoinEngines["RDBMS Physical Join Execution Algorithms"]
        NL["Nested Loop Join\nBest for small outer table + indexed inner table (O(N log M))"]
        HJ["Hash Join\nBest for large unindexed tables, equality joins (O(N + M))"]
        SMJ["Sort-Merge Join\nBest when both inputs are already sorted by join key (O(N + M))"]
    end
```

4. Simple Example: INNER vs LEFT JOIN

```
-- INNER JOIN: Only customers who have at least one account
SELECT c.customer_id, c.first_name, a.account_number
FROM customers c
INNER JOIN accounts a ON c.customer_id = a.customer_id;

-- LEFT JOIN: All customers, including newly registered customers with no accounts yet
SELECT c.customer_id, c.first_name, a.account_number
FROM customers c
LEFT JOIN accounts a ON c.customer_id = a.customer_id;
```

5. Real-World Banking Example: Detecting Unfunded Accounts & Dormant Customers

A customer onboarding compliance officer needs a report of all customers whose KYC is verified, but who have either zero accounts or whose accounts hold zero total balance.

```
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    c.phone,
    COUNT(a.account_id) AS total_accounts,
    COALESCE(SUM(a.balance), 0.00) AS total_liquidity
FROM customers c
LEFT JOIN accounts a ON c.customer_id = a.customer_id
WHERE c.kyc_status = 'VERIFIED'
GROUP BY c.customer_id, c.first_name, c.last_name, c.phone
HAVING COUNT(a.account_id) = 0 OR SUM(a.balance) = 0.00;
```

6. Code: Multi-Table Financial Ledger Join with Self-Join

```
-- Comprehensive Transaction Audit: Sender details, Receiver details, and Card info
SELECT
    t.transaction_reference,
    t.created_at,
    t.amount,
    t.payment_channel,
    -- Sender Information
    sender_cust.first_name || ' ' || sender_cust.last_name AS sender_name,
    sender_acc.account_number AS sender_account,
    -- Receiver Information
    receiver_cust.first_name || ' ' || receiver_cust.last_name AS receiver_name,
    receiver_acc.account_number AS receiver_account
FROM transactions t
LEFT JOIN accounts sender_acc ON t.from_account_id = sender_acc.account_id
LEFT JOIN customers sender_cust ON sender_acc.customer_id = sender_cust.customer_id
LEFT JOIN accounts receiver_acc ON t.to_account_id = receiver_acc.account_id
LEFT JOIN customers receiver_cust ON receiver_acc.customer_id = receiver_cust.customer_id
WHERE t.status = 'SUCCESS'
ORDER BY t.created_at DESC;
```

7. How It Works Internally: Physical Join Algorithms

1. Nested Loop Join

- The engine loops over the outer table row by row, and for each row looks up matching keys in the inner table.
- If the inner table join column has a B-Tree index, complexity is $O(N \log M)$. If unindexed, it degrades to quadratic $O(N * M)$ Cartesian scans.

2. Hash Join

- The query planner picks the smaller table (build input) and builds an in-memory hash table on the join key.
- It then scans the larger table (probe input) once, hashing each key and probing the in-memory hash table for matches.
- **Complexity:** $O(N + M)$ time.
- **Memory footprint:** If the build table exceeds database working memory (`work_mem` in PostgreSQL), the engine spills hash partitions to disk (Grace Hash Join), causing dramatic I/O slowdowns.

3. Sort-Merge Join

- Both tables are sorted by the join key (or scanned in order via existing B-Tree indexes).
- Two pointers walk both streams simultaneously, merging matches in a single pass.
- **Complexity:** $O(N \log N + M \log M)$ if unsorted, but $O(N + M)$ if pre-indexed.

8. Common Mistakes

1. **Filtering the Right Table in the `WHERE` Clause Instead of the `ON` Clause in a `LEFT JOIN`:** `sql --`
 WRONG: Accidentally converts the LEFT JOIN into an INNER JOIN! `SELECT c.customer_id, a.balance`
`FROM customers c LEFT JOIN accounts a ON c.customer_id = a.customer_id WHERE a.status =`
`'ACTIVE';` -- Filters out NULL rows where customer had no accounts!

-- CORRECT: Keep filter in the ON predicate to preserve customers with no accounts `SELECT`
`c.customer_id, a.balance FROM customers c LEFT JOIN accounts a ON c.customer_id = a.customer_id`
`AND a.status = 'ACTIVE';`

2. ****Missing COALESCE with SUM() in Outer Joins:**** `SUM(NULL)` returns NULL, not 0.00. Always wrap aggregate expressions in `COALESCE(SUM(balance), 0)`.
3. ****Unintended Cartesian Products:**** Forgetting the `ON` condition creates a CROSS JOIN, multiplying rows exponentially and crashing database memory.

9. Performance / Complexity Matrix

Join Type / Algorithm	Best Scenario	Time Complexity	Memory Requirements
Nested Loop (Indexed)	Small outer table, indexed inner key	$O(N \log M)$	$O(1)$
Nested Loop (Unindexed)	Very small tables only	$O(N * M)$	$O(1)$
Hash Join	Large unindexed tables, equality joins	$O(N + M)$	$O(M)$ (<code>work_mem</code>)
Sort-Merge Join	Both inputs already sorted on join key	$O(N + M)$	$O(1)$

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is the difference between an `INNER JOIN` and a `LEFT JOIN` ?

Medium

- **Q:** How does putting a filter condition in the `ON` clause differ from putting it in the `WHERE` clause during a `LEFT JOIN` ?

Hard

- **Q:** You observe a query joining `transactions` (100M rows) and `accounts` (10M rows) suddenly slowing down from 200ms to 45 seconds after a traffic surge. `EXPLAIN` shows the planner shifted from a Hash Join to a disk-based external merge join. Why did this happen, and how do you fix it?

11. Follow-up Questions from Interviewer

- "What is `work_mem` in PostgreSQL, and what happens when a Hash Join's hash table exceeds `work_mem` ?"
- "Can a Hash Join be used for non-equality joins like `ON a.date ≥ b.start_date AND a.date ≤ b.end_date` ?" (Answer: No! Hash tables require exact equality for bucket hashing; range joins must use Nested Loop or Merge Joins).

12. Model Answer: Hash Join Memory Spilling & Remediation

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "Why would a join query suddenly slow down by orders of magnitude when table size grows slightly?"

Model Answer: "The most common culprit in production relational databases (like PostgreSQL) is **hash table memory spilling**.

During an in-memory Hash Join, the optimizer loads the smaller table's join keys into RAM bounded by the session parameter `work_mem`.

If the build table size exceeds `work_mem`, the database can no longer perform a one-pass in-memory hash join. It falls back to a **multi-batch hybrid hash join** (or external sort-merge join), writing intermediate buckets to temporary disk files. Disk I/O is three orders of magnitude slower than RAM, causing the sudden latency cliff from 200ms to 45 seconds.

Remediation Strategy: 1. Increase `work_mem` for the specific reporting session or service role (`SET work_mem = '128MB';`). 2. Ensure foreign key columns have B-Tree indexes so the planner can consider an index-nested loop join if only a filtered subset is needed. 3. Filter the build table early before joining (predicate pushdown)."

13. Practical Exercise (Run on `banking.db`)

Execute the following query to find any customers who have transactions where they sent money to themselves across their own different accounts:

```
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    t.transaction_reference,
    t.amount,
    from_acc.account_number AS debited_account,
    to_acc.account_number AS credited_account
FROM transactions t
JOIN accounts from_acc ON t.from_account_id = from_acc.account_id
JOIN accounts to_acc ON t.to_account_id = to_acc.account_id
JOIN customers c ON from_acc.customer_id = c.customer_id
WHERE from_acc.customer_id = to_acc.customer_id
    AND from_acc.account_id ≠ to_acc.account_id;
```

14. Quick Revision

- **INNER JOIN** keeps only matching rows; **LEFT JOIN** keeps all left rows and fills right mismatches with **NULL**.
- In **LEFT JOIN**, right table conditions in **WHERE** negate the outer join, turning it into an **INNER JOIN**.
- Physical joins: Nested Loop (indexed small sets), Hash Join (large unsorted sets, equality only), Sort-Merge Join (sorted inputs).

- Non-equality joins cannot use Hash Joins.

15. Interview Checklist

- [] Understands the difference between `ON` and `WHERE` filtering in outer joins.
- [] Can name and contrast Nested Loop, Hash Join, and Sort-Merge Join.
- [] Knows that Hash Joins require equality predicates (`=`).
- [] Explains `work_mem` and disk spill during hash joins.

CHAPTER 02 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Relational keys (Primary Key, Foreign Key).
- Basic SQL syntax (`SELECT ... FROM ... WHERE ...`).
- INNER JOIN: Returns records where the join predicate matches in *both* tables.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

SQL Subqueries, Correlated Execution & EXISTS vs IN

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

SQL Subqueries, Correlated Execution & EXISTS vs IN

1. Why This Matters

Subqueries appear frequently in technical screenings and production query tuning at IDFC FIRST Bank. A poorly constructed subquery can transform an $O(N)$ linear query into an $O(N^2)$ correlated loop executed for every row in a multi-million-row financial ledger. Understanding the difference between scalar subqueries, correlated subqueries, and why `EXISTS` is safer and faster than `IN` when dealing with `NULL` values is a crucial differentiator for a 3+ YoE developer.

2. Prerequisites

- Basic SQL syntax (`SELECT` , `WHERE` , `GROUP BY`).
- Relational joins and Cartesian concepts.

3. Concept

Taxonomy of Subqueries

- Scalar Subquery:** Returns a single value (1 row, 1 column). Can be used anywhere an expression is expected (e.g., `SELECT` , `WHERE`).
- Multi-Row Subquery:** Returns multiple rows (1 column). Evaluated with set operators (`IN` , `NOT IN` , `ANY` , `ALL`).
- Correlated Subquery:** References columns from the outer query. It cannot be evaluated independently; the database engine conceptually evaluates the inner query *once for every candidate row* produced by the outer query.
- Non-Correlated Subquery:** Independent of the outer query. Evaluated once, and its result is cached and reused.

```

flowchart TD
    subgraph NonCorrelated["Non-Correlated Subquery (Evaluated Once)"]
        Inner1["Inner Query: SELECT AVG(balance) FROM accounts"] -->|Single Scalar Value| Outer1["Outer Query: WHERE balance > [Value]"]
    end

    subgraph Correlated["Correlated Subquery (Evaluated N Times)"]
        OuterRow["For each row in accounts a1"] --> Inner2["Evaluate: SELECT AVG(a2.balance) WHERE a2.branch = a1.branch"]
        Inner2 -->|Comparison Result| Decision["Keep or Filter Row"]
    end

```

4. Simple Example: Scalar vs Correlated Subquery

```

-- Non-Correlated Scalar: Accounts with balance above the overall bank average
SELECT account_number, balance
FROM accounts
WHERE balance > (SELECT AVG(balance) FROM accounts);

-- Correlated: Accounts with balance above their own branch's average
SELECT a1.account_number, a1.branch_code, a1.balance
FROM accounts a1
WHERE a1.balance > (
    SELECT AVG(a2.balance)
    FROM accounts a2
    WHERE a2.branch_code = a1.branch_code
);

```

5. Real-World Banking Example: Finding Customers with High Fraud Risk

Flag customers who have initiated more than 2 failed transactions in the last 24 hours using **EXISTS** :

```

SELECT c.customer_id, c.first_name, c.last_name, c.phone
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM transactions t
    JOIN accounts a ON t.from_account_id = a.account_id
    WHERE a.customer_id = c.customer_id
    AND t.status = 'FAILED'
    AND t.created_at ≥ DATETIME('now', '-1 day')
    GROUP BY a.customer_id
    HAVING COUNT(*) ≥ 2
);

```

6. Code: Complex Subquery Optimization on `banking.db`

```
-- 1. Unlinked Accounts: Customers who have never performed any transactions
-- Optimized using NOT EXISTS
SELECT c.customer_id, c.first_name, c.email
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM accounts a
    JOIN transactions t ON (t.from_account_id = a.account_id OR t.to_account_id = a.account_id)
    WHERE a.customer_id = c.customer_id
);

-- 2. Finding the single largest transaction for each account
SELECT t1.from_account_id, t1.transaction_reference, t1.amount
FROM transactions t1
WHERE t1.amount = (
    SELECT MAX(t2.amount)
    FROM transactions t2
    WHERE t2.from_account_id = t1.from_account_id
);
```

7. How It Works Internally: EXISTS vs IN & The Three-Valued Logic Trap

Why `NOT IN` Fails with NULLs

In SQL, boolean logic is three-valued: `TRUE`, `FALSE`, and `UNKNOWN` (represented by `NULL`). When you run:

```
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM accounts);
```

If the subquery returns even a single row with `NULL`, the condition evaluates to: `customer_id ≠ 1 AND customer_id ≠ 2 AND customer_id ≠ NULL`. Because `x ≠ NULL` is `UNKNOWN`, the entire `AND` conjunction evaluates to `UNKNOWN`, and the query returns **zero rows**, silently masking all valid results!

Why `EXISTS` is Superior

`EXISTS` tests for the existence of at least one matching row. It short-circuits to `TRUE` the moment the engine finds the first matching row without scanning remaining pages, and it is completely immune to `NULL` column values.

8. Common Mistakes

1. **Writing `SELECT *` inside `EXISTS`**: While modern optimizers rewrite `EXISTS (SELECT * ...)` to `EXISTS (SELECT 1 ...)`, always write `SELECT 1` as standard senior engineering practice to signal that no column data projection is needed.

2. **Accidental $O(N^2)$ Correlated Subqueries in `SELECT` Lists:** ``sql -- BAD: Runs an index scan or table scan on transactions for every row in customers
`SELECT c.customer_id, (SELECT COUNT(*) FROM transactions t JOIN accounts a ON t.from_account_id = a.account_id WHERE a.customer_id = c.customer_id) AS tx_count FROM customers c;`

-- GOOD: Rewrite as a single LEFT JOIN with GROUP BY ($O(N + M)$)
`SELECT c.customer_id, COUNT(t.transaction_id) AS tx_count FROM customers c LEFT JOIN accounts a ON c.customer_id = a.customer_id LEFT JOIN transactions t ON a.account_id = t.from_account_id GROUP BY c.customer_id;```

9. Performance / Complexity Matrix

Technique	Execution Complexity	Optimizer Capability
Non-Correlated Subquery	$O(N + M)$	Easily cached or transformed into a Hash/Merge Join
Correlated Subquery (Indexed)	$O(N \log M)$	Evaluated via indexed nested loop
Correlated Subquery (Unindexed)	$O(N * M)$	Devastating full table scan per outer row
<code>EXISTS</code> with Index	$O(\log M)$ per row	Short-circuits on first row match
Subquery Rewritten to <code>JOIN</code>	$O(N + M)$	Fully parallelizable, enables Hash Joins

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is the difference between a correlated subquery and a non-correlated subquery?

Medium

- **Q:** Why can `NOT IN (SELECT col FROM ...)` return an empty set if the inner query contains `NULL` values? How does `NOT EXISTS` prevent this?

Hard

- **Q:** How do modern database query optimizers perform **Subquery Unnesting** (decorrelation), and when does decorrelation fail, forcing the engine into a row-by-row loop?

11. Follow-up Questions from Interviewer

- "If you write a correlated subquery in the `WHERE` clause, does the database literally execute the subquery for every outer row?" (Answer: Not necessarily! Modern RDBMS optimizers attempt subquery unnesting to transform correlated subqueries into semi-joins or anti-joins. However, if aggregate functions or `LIMIT` clauses prevent unnesting, it falls back to row-by-row nested loop execution).
- "How does a Common Table Expression (CTE) compare to a derived subquery in PostgreSQL 12+ vs older versions?"

12. Model Answer: Subquery Unnesting & Semi-Joins

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "Can you explain what a Semi-Join is and how the optimizer uses it for `EXISTS` subqueries?"

Model Answer: "A **Semi-Join** is an internal relational operator used by query engines to evaluate `WHERE EXISTS (subquery)` or `WHERE x IN (subquery)` .

Unlike a standard `INNER JOIN` which multiplies rows when the right table has multiple matches, a Semi-Join returns each row from the outer table at most once, immediately stopping the probe on the inner table as soon as the first match is encountered.

Through a process called **Subquery Unnesting**, the query optimizer lifts the subquery into the main query plan and executes it as a Hash Semi-Join or Merge Semi-Join. This avoids the theoretical $O(N * M)$ row-by-row execution and achieves optimal $O(N + M)$ performance without application developers needing to manually rewrite the query."

13. Practical Exercise (Run on `banking.db`)

Execute a query to find all accounts whose balance is strictly above the average balance of all accounts within their own account type (`SAVINGS` vs `CURRENT` vs `SALARY`):

```
SELECT a1.account_number, a1.account_type, a1.balance, ROUND(avg_table.avg_bal, 2) AS
category_avg
FROM accounts a1
JOIN (
    SELECT account_type, AVG(balance) AS avg_bal
    FROM accounts
    GROUP BY account_type
) avg_table ON a1.account_type = avg_table.account_type
WHERE a1.balance > avg_table.avg_bal;
```

14. Quick Revision

- Non-correlated subqueries evaluate once; correlated subqueries depend on the outer row.

- `NOT IN` fails silently when `NULL` values exist due to three-valued logic.
- Always prefer `EXISTS` / `NOT EXISTS` or outer joins over `IN` / `NOT IN`.
- Semi-joins return outer rows without duplicate row inflation.
- Avoid correlated subqueries inside `SELECT` projection lists.

15. Interview Checklist

- [] Understands the three-valued logic trap of `NOT IN` with `NULL`.
- [] Can explain how an optimizer short-circuits `EXISTS`.
- [] Understands subquery unnesting into semi-joins.
- [] Can rewrite a correlated scalar subquery into a join with `GROUP BY`.

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Basic SQL syntax ('SELECT', 'WHERE', 'GROUP BY').
- Relational joins and Cartesian concepts.
- Q: What is the difference between a correlated subquery and a non-correlated subquery?

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

SQL Window Functions for Financial Ledgers & Analytics

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

SQL Window Functions for Financial Ledgers & Analytics

1. Why This Matters

Window functions are the single most asked SQL topic in 3+ YoE banking tech interviews at IDFC FIRST Bank. They are essential for computing running account balances, calculating month-over-month growth, finding the top N transactions per customer, and detecting fraud velocity spikes without collapsing rows into a single summary line like `GROUP BY`.

2. Prerequisites

- Basic SQL aggregations (`SUM`, `COUNT`, `AVG`).
- Understanding of the `ORDER BY` clause.

3. Concept

Window Function Anatomy

Unlike `GROUP BY`, which collapses multiple rows into a single aggregated row, a **Window Function** computes a calculation across a set of table rows that are related to the current row, while **retaining the identity of every single individual row**.

`FUNCTION() OVER (PARTITION BY col1 ORDER BY col2 [ROWS/RANGE FRAME])`

```
flowchart TD
    subgraph Windowing["Window Partitioning & Framing"]
        T["Input Table Rows"] --> P1["Partition 1: Account 101"]
        T --> P2["Partition 2: Account 103"]
        P1 --> O1["Sort by created_at"]
        O1 --> F1["Frame: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW"]
        F1 --> R1["Running Balance Computed per Row"]
    end
```

Key Window Function Categories

1. Ranking Functions:

2. `ROW_NUMBER()` : Unique sequential integer (1, 2, 3, 4). Ties get distinct numbers arbitrarily.
3. `RANK()` : Sequential integer, but ties receive the same rank, and subsequent ranks are skipped (1, 2, 2, 4).
4. `DENSE_RANK()` : Sequential integer; ties receive the same rank, and subsequent ranks are **not** skipped (1, 2, 2, 3).
5. **Value / Navigation Functions:**
6. `LAG(col, offset)` : Accesses data from a previous row within the partition without a self-join.
7. `LEAD(col, offset)` : Accesses data from a subsequent row within the partition.
8. **Aggregate Window Functions:**
9. `SUM() OVER (...)` , `AVG() OVER (...)` , `COUNT() OVER (...)` .

4. Simple Example: ROW_NUMBER vs RANK vs DENSE_RANK

```
SELECT
  customer_id,
  amount,
  ROW_NUMBER() OVER (ORDER BY amount DESC) AS row_num,
  RANK() OVER (ORDER BY amount DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
FROM transactions;
```

5. Real-World Banking Example: Real-Time Account Statement with Running Balance

Generating an official bank account statement showing each transaction along with the cumulative running balance:

```
SELECT
  transaction_id,
  from_account_id AS account_id,
  created_at,
  transaction_type,
  amount,
  -- Subtract withdrawals/transfers, add deposits
  SUM(CASE
    WHEN transaction_type = 'DEPOSIT' THEN amount
    ELSE -amount
  END) OVER (
    PARTITION BY from_account_id
    ORDER BY created_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_balance
FROM transactions
WHERE status = 'SUCCESS'
ORDER BY from_account_id, created_at;
```

6. Code: Top 2 Transactions per Customer Using CTE & DENSE_RANK

```
WITH RankedTransactions AS (
  SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    t.transaction_reference,
    t.amount,
    t.payment_channel,
    t.created_at,
    DENSE_RANK() OVER (
      PARTITION BY c.customer_id
      ORDER BY t.amount DESC
    ) AS rank_by_amount
  FROM customers c
  JOIN accounts a ON c.customer_id = a.customer_id
  JOIN transactions t ON a.account_id = t.from_account_id
  WHERE t.status = 'SUCCESS'
)
SELECT *
FROM RankedTransactions
WHERE rank_by_amount ≤ 2
ORDER BY customer_id, rank_by_amount;
```

7. How It Works Internally: Window Frames & Default Framing Pitfall

The Default Framing Trap

When you specify **ORDER BY** inside **OVER ()** without an explicit frame clause: **DEFAULT FRAME = RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**

- **RANGE vs ROWS** :
- **ROWS** counts physical rows (e.g., exactly 1 preceding row).
- **RANGE** treats all rows with **identical order values as peers**, buffering and summing them all together!
- In large transaction tables, **RANGE** forces the database engine to check for peer duplicates, which prevents index streaming and significantly slows down execution.
- **Senior Practice:** Always specify **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** for deterministic running totals and maximum performance.

8. Common Mistakes

1. **Attempting to Filter Window Functions in the WHERE Clause:** `sql -- SYNTAX ERROR: Window functions are NOT allowed in WHERE! SELECT account_id, amount, ROW_NUMBER() OVER (ORDER BY amount DESC) as rn FROM transactions WHERE ROW_NUMBER() OVER (ORDER BY amount DESC) ≤ 3;`
Reason: In the SQL logical execution pipeline, **WHERE** is evaluated at Step 3, whereas Window Functions are computed during the **SELECT** evaluation at Step 6. You must wrap the window query inside a **Common Table Expression (CTE)** or derived subquery to filter on its output.

2. **Confusing `RANK()` with `DENSE_RANK()`** : If an interviewer asks for the "top 3 salaries" or "top 3 transaction amounts", using `RANK()` can return ranks `1, 2, 2, 4` —accidentally omitting the 3rd highest value! Always use `DENSE_RANK()` .

9. Performance / Complexity Matrix

Window Function	Required Sort	Memory Buffer Type	Complexity
<code>ROW_NUMBER()</code>	<code>ORDER BY</code> column	Streaming (Scalar counter)	O(N log N) (or O(N) with index)
<code>DENSE_RANK()</code>	<code>ORDER BY</code> column	Scalar peer comparator	O(N log N)
<code>LAG()</code> / <code>LEAD()</code>	<code>ORDER BY</code> column	Sliding offset ring buffer	O(N log N)
<code>SUM() OVER (ROWS...)</code>	<code>ORDER BY</code> column	Cumulative running accumulator	O(N log N)
<code>SUM() OVER (RANGE...)</code>	<code>ORDER BY</code> column	Multi-row peer buffer	Slower than <code>ROWS</code>

● **Easy Question:** `ROW_NUMBER()` vs `RANK()` vs `DENSE_RANK()`

Q: What is the precise behavioral difference between `ROW_NUMBER()` , `RANK()` , and `DENSE_RANK()` when handling duplicate (tied) values? Illustrate with a concrete example.

Complete Answer: When ordering rows that contain identical sort values (ties): 1. `ROW_NUMBER()` : Assigns a strictly unique, contiguous sequential integer starting at 1 to every row within the partition, breaking ties arbitrarily non-deterministically. 2. `RANK()` : Assigns identical rank numbers to tied rows, but **skips subsequent ranks** to account for the count of duplicates (gap ranking). 3. `DENSE_RANK()` : Assigns identical rank numbers to tied rows, but **never skips numbers** (no gaps in sequence).

Comparative Example (Salaries of 100k, 100k, 80k, 60k):

Employee	Salary	<code>ROW_NUMBER()</code>	<code>RANK()</code>	<code>DENSE_RANK()</code>	Explanation
Priya	₹1,00,000	1	1	1	Tied for 1st place
Rahul	₹1,00,000	2	1	1	Tied for 1st place
Amit	₹80,000	3	3	2	<code>RANK</code> skips 2;
Sneha	₹60,000	4	4	3	<code>DENSE_RANK</code> assigns 2 Contiguous numbering continues

- **Interview Tip:** For "Nth highest salary/transaction", always use `DENSE_RANK()` . Using `RANK()` could cause `WHERE r = 2` to return 0 rows if there is a tie for first place!

● **Medium Question:** Second Highest Transaction Per Branch Without `LIMIT` / `OFFSET`

Q: Write an enterprise SQL query to find the second highest transaction amount for every bank branch. You cannot use `LIMIT` or `OFFSET` because the branch count is dynamic.

Complete Answer: Use a Common Table Expression (CTE) with `DENSE_RANK()` partitioned by `branch_id` and ordered by `amount DESC` :

```
WITH RankedBranchTransactions AS (  
    SELECT  
        branch_id,  
        account_id,  
        transaction_reference,  
        amount,  
        DENSE_RANK() OVER (  
            PARTITION BY branch_id  
            ORDER BY amount DESC  
        ) AS rank_in_branch  
    FROM transactions  
    WHERE status = 'SUCCESS'  
)  
SELECT  
    branch_id,  
    account_id,  
    transaction_reference,  
    amount AS second_highest_amount  
FROM RankedBranchTransactions  
WHERE rank_in_branch = 2;
```

- **Why this is optimal:** Handles ties gracefully (if two customers in Branch 1 tied for the highest amount of ₹50,000, the next distinct highest amount of ₹40,000 will correctly be identified as rank 2).

● Hard Question: Detecting 3 Consecutive Increasing Transactions

Q: In a banking fraud detection system, write a query using `LAG()` to identify accounts that made 3 or more consecutive transactions where each transaction amount was strictly greater than the preceding transaction.

Complete Answer: Use `LAG(amount, 1)` and `LAG(amount, 2)` partitioned by `account_id` and ordered by `created_at` :

```

WITH TxSeries AS (
    SELECT
        account_id,
        transaction_reference,
        created_at,
        amount,
        LAG(amount, 1) OVER (
            PARTITION BY account_id
            ORDER BY created_at
        ) AS prev_amount_1,
        LAG(amount, 2) OVER (
            PARTITION BY account_id
            ORDER BY created_at
        ) AS prev_amount_2
    FROM transactions
    WHERE status = 'SUCCESS'
)
SELECT
    account_id,
    transaction_reference AS third_tx_ref,
    created_at,
    prev_amount_2 AS first_txn_amount,
    prev_amount_1 AS second_txn_amount,
    amount AS third_txn_amount
FROM TxSeries
WHERE prev_amount_2 IS NOT NULL
    AND prev_amount_1 > prev_amount_2
    AND amount > prev_amount_1;

```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "Why can't window functions be used directly in the `WHERE` or `HAVING` clause? Why must we wrap them in a subquery or CTE?"

Complete Answer: – **The SQL Logical Query Processing Order:** The SQL standard mandates a strict logical order of evaluation: 1. `FROM` (table joins and cartesian products) 2. `WHERE` (row-level filtering) 3. `GROUP BY` (aggregating into group buckets) 4. `HAVING` (filtering group buckets) 5. `SELECT` (evaluating expressions and **Window Functions**) 6. `DISTINCT` (removing duplicate output rows) 7. `ORDER BY` (sorting final output) 8. `LIMIT` / `OFFSET` (paging) – **Why It Fails in `WHERE` / `HAVING`:** – Window functions operate over the **window of rows that have already survived the `WHERE` and `HAVING` filters!** – If a window function were permitted in `WHERE`, it would produce a circular logical dependency: you cannot evaluate a window function across the filtered dataset before the dataset has been filtered. – **The Solution:** Evaluate the window function in a `SELECT` clause within a **CTE or derived subquery** (Step 5 of inner query), then filter on its calculated column in the outer query's `WHERE` clause (Step 2 of outer query).

Follow-up 2

Interviewer: "How does a database engine physically execute `PARTITION BY` and `ORDER BY` in window functions? Does it always sort the dataset, and how can we optimize it with B-Tree indexes?"

Complete Answer: 1. **Physical Execution Mechanics:** - The database optimizer introduces a **WindowAgg** execution node. - To compute window functions efficiently without maintaining infinite lookback buffers, the engine requires that all rows belonging to the same partition appear **contiguously** in memory. - Therefore, the optimizer injects a **Sort Node** before the WindowAgg node: **Sort Key:** `partition_col ASC, order_col ASC`. - As the engine streams rows sequentially: - When `partition_col` changes, it resets window registers. - Within the partition, it maintains running tallies or sliding frames in linear O(N) streaming time. 2. **Index Optimization (Eliminating the Expensive Sort):** - Sorting millions of rows in memory (**SortMethod:** `quicksort` or on-disk `external merge disk`) is CPU- and I/O-intensive. - If you create a **Composite B-Tree Index:** `sql CREATE INDEX idx_tx_account_time ON transactions (from_account_id, created_at);` - The B-Tree index stores table rows *already sorted* by `from_account_id` and then by `created_at` ! - The optimizer completely eliminates the Sort node and performs a direct **Index Scan**, streaming rows straight into the WindowAgg node with **zero sorting overhead** (reducing query latency from 1,200ms to 4ms on large tables).

13. Practical Exercise (Run on `banking.db`)

Execute the running balance query on `banking.db` :

```
sqlite3 banking.db
```

```
SELECT
  t.transaction_reference,
  t.from_account_id,
  t.amount,
  t.created_at,
  SUM(t.amount) OVER (
    PARTITION BY t.from_account_id
    ORDER BY t.created_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_spent
FROM transactions t
WHERE t.from_account_id IS NOT NULL;
```

14. Quick Revision

- Window functions retain row identity; `GROUP BY` collapses rows.
- Syntax: `FUNCTION() OVER (PARTITION BY ... ORDER BY ... [FRAME])`.
- `ROW_NUMBER` (unique integers), `RANK` (skips ties), `DENSE_RANK` (no skipped ties).
- Never use window functions in `WHERE` directly; wrap them in a CTE.
- Explicitly use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` to avoid default `RANGE` peer comparison overhead.

15. Interview Checklist

- [] Clearly explains `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` with numerical examples.
- [] Understands why window functions cannot be placed in `WHERE` or `HAVING` .
- [] Can write running total queries with correct frame definitions.
- [] Can use `LAG()` and `LEAD()` for time-series gap analysis.



CHAPTER 04 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Basic SQL aggregations (`SUM`, `COUNT`, `AVG`).
- Understanding of the `ORDER BY` clause.
- `ROW\_NUMBER()`: Unique sequential integer (1, 2, 3, 4).
Ties get distinct numbers arbitrarily.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

Common Table Expressions (CTEs) & Recursive CTEs

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Common Table Expressions (CTEs) & Recursive CTEs

1. Why This Matters

In enterprise banking applications, SQL queries can quickly become unmaintainable when multiple levels of business logic, filtering, and window calculations are chained together. At IDFC FIRST Bank, developers use **Common Table Expressions (CTEs)** to construct clean, modular data pipelines. In technical interviews, writing a clean recursive CTE (to traverse hierarchical corporate accounts, branch networks, or generate continuous date ranges for financial reporting) signals senior-level SQL mastery.

2. Prerequisites

- Basic SQL queries and Subqueries.
- Set operations (`UNION` vs `UNION ALL`).

3. Concept

What is a Common Table Expression?

A **CTE** is a temporary, named result set that exists only within the execution scope of a single SQL statement (`SELECT` , `INSERT` , `UPDATE` , or `DELETE`). It acts like an in-memory view defined inline.

```
WITH CTE_Name AS (  
    -- Anchor or Subquery logic  
    SELECT column1, column2 FROM table_name WHERE condition  
)  
SELECT * FROM CTE_Name;
```

Recursive CTEs

A **Recursive CTE** references itself. It consists of three parts: 1. **Anchor Member**: The initial query that forms the base result set (e.g., top-level root branch). 2. **Recursive Member**: A query that joins the CTE back to the underlying table, executed repeatedly until it returns no new rows. 3. **`UNION ALL` Operator**: Glues the anchor and recursive iterations together.

flowchart TD

```
A["1. Execute Anchor Query (Base Level: Root Nodes)"] --> U["UNION ALL"]
U --> R["2. Execute Recursive Query (Join CTE on parent_id = id)"]
R --> C{"Did recursive query return new rows?"}
C -->|Yes| U
C -->|No| F["3. Terminate & Return Complete Hierarchy"]
```

4. Simple Example: Standard CTE vs Subquery Readability

```
-- Standard CTE: Modular and readable
WITH HighValueTransfers AS (
    SELECT transaction_id, from_account_id, amount
    FROM transactions
    WHERE amount ≥ 50000.00 AND status = 'SUCCESS'
)
SELECT from_account_id, COUNT(*) AS large_transfer_count, SUM(amount) AS total_volume
FROM HighValueTransfers
GROUP BY from_account_id;
```

5. Real-World Banking Example: Generating Missing Dates for Continuous Cash Flow

Financial charts require showing daily balances even on days with **zero transactions**. If a query only groups by `transactions.created_at`, days without transactions are skipped entirely! Using a **Recursive CTE**, we generate a complete date calendar and `LEFT JOIN` it against transactions:

```
-- Generate all dates in March 2026 and display daily transaction volume
WITH RECURSIVE DateCalendar AS (
    -- Anchor: Start Date
    SELECT DATE('2026-03-01') AS calendar_date
    UNION ALL
    -- Recursive Step: Add 1 day until End Date
    SELECT DATE(calendar_date, '+1 day')
    FROM DateCalendar
    WHERE calendar_date < '2026-03-15'
)
SELECT
    d.calendar_date,
    COUNT(t.transaction_id) AS total_transactions,
    COALESCE(SUM(t.amount), 0.00) AS total_volume
FROM DateCalendar d
LEFT JOIN transactions t ON DATE(t.created_at) = d.calendar_date AND t.status = 'SUCCESS'
GROUP BY d.calendar_date
ORDER BY d.calendar_date;
```

6. Code: Corporate Hierarchy Traversal Using Recursive CTE

Suppose a commercial banking customer has parent corporate entities, subsidiaries, and division accounts:

```

-- Conceptual schema:
-- corporate_entities (entity_id, entity_name, parent_entity_id)

WITH RECURSIVE EntityHierarchy AS (
    -- Anchor: Find the root parent company (e.g., entity_id = 1)
    SELECT entity_id, entity_name, parent_entity_id, 0 AS hierarchy_level
    FROM corporate_entities
    WHERE entity_id = 1

    UNION ALL

    -- Recursive Member: Find all child subsidiaries
    SELECT c.entity_id, c.entity_name, c.parent_entity_id, eh.hierarchy_level + 1
    FROM corporate_entities c
    INNER JOIN EntityHierarchy eh ON c.parent_entity_id = eh.entity_id
)
SELECT * FROM EntityHierarchy
ORDER BY hierarchy_level, entity_name;

```

7. How It Works Internally: CTE Materialization Fences (PostgreSQL 12+)

Optimization Fence History

- In older database versions (PostgreSQL < 12), CTEs acted as **strict optimization fences**. The database computed the CTE once, materialized it in memory (or spilled to temporary disk), and prevented the optimizer from pushing outer **WHERE** predicates into the CTE.
- In modern engines (PostgreSQL 12+, MySQL 8+, SQLite 3.35+), CTEs are **inlined** by default. The optimizer treats a CTE as a subquery, allowing predicate pushdown.
- Controlling Materialization:** ``sql -- Force materialization (prevent re-evaluation if expensive): WITH ExpensiveCalc AS MATERIALIZED (...)

-- Force inlining (allow index predicate pushdown): WITH InlinedFilter AS NOT MATERIALIZED (...)

8. Common Mistakes

- Infinite Loops in Recursive CTEs:** Forgetting a termination condition in the **WHERE** clause of the recursive member causes an infinite recursion loop, terminating only when the database hits **max_recursion_depth** (default 100 in SQL Server, 1000 in MySQL).
- Using **UNION** Instead of **UNION ALL** in Recursive CTEs:** In recursive definitions, standard SQL requires **UNION ALL**. Using **UNION** forces expensive deduplication sorting across each recursive step.
- Overusing CTEs for Trivial Lookups:** Defining 10 trivial one-line CTEs when a clean join or where clause suffices adds parsing overhead and reduces query clarity.

9. Performance / Complexity Matrix

Technique	Execution Model	Memory Impact
Inlined CTE (Default)	Blended into outer query plan	O(1) extra memory (streaming)
Materialized CTE	Computed once and cached in RAM	O(K) where K is CTE row count
Recursive CTE	Breadth-First loop over working table	O(depth * breadth)
Deeply Nested Subqueries	Hard to read; optimizer rewrites	Equivalent to inlined CTE

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is a Common Table Expression (CTE) and how does it differ from a database View?

Medium

- **Q:** What is the difference between `UNION` and `UNION ALL` inside a recursive CTE?

Hard

- **Q:** Explain what an "Optimization Fence" was in early PostgreSQL versions regarding CTEs. When would you explicitly use `AS MATERIALIZED` versus `AS NOT MATERIALIZED` in modern database engines?

11. Follow-up Questions from Interviewer

- "If your recursive CTE hierarchy has a circular reference (e.g., Entity A is parent of B, but B is marked as parent of A), how do you prevent an infinite recursion crash?" (Answer: Track an array or path string of visited IDs: `WHERE c.entity_id ≠ ALL(eh.visited_path)`).

12. Model Answer: Preventing Infinite Recursive Loops

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "How do you detect and prevent cycles during a recursive hierarchy traversal in SQL?"

Model Answer: "To prevent infinite recursion caused by corrupted circular data (such as Account A owning Account B, which owns Account A), we maintain an **ancestry path array** inside the CTE.

1. In the Anchor query, we initialize an array containing the root ID: `ARRAY[entity_id] AS path`.
2. In the Recursive query, before joining a child row, we verify that the child's `entity_id` is NOT already in `path`: `sql WHERE c.entity_id ≠ ALL(eh.path)`
3. We append the child's ID to the path: `path || c.entity_id`.
4. If a cycle occurs, the `WHERE` condition evaluates to false, gracefully terminating the branch without crashing the database server."

13. Practical Exercise (Run on `banking.db`)

Execute the recursive date calendar query on `banking.db` to verify your environment supports recursive CTEs:

```
sqlite3 banking.db
```

```
WITH RECURSIVE DateCalendar AS (  
  SELECT DATE('2026-03-01') AS calendar_date  
  UNION ALL  
  SELECT DATE(calendar_date, '+1 day')  
  FROM DateCalendar  
  WHERE calendar_date < '2026-03-07'  
)  
SELECT * FROM DateCalendar;
```

14. Quick Revision

- CTEs are defined using `WITH ... AS (...)`.
- Unlike Views, CTEs exist only for the duration of the query.
- Recursive CTEs require an Anchor, `UNION ALL`, and a Recursive Member that joins back to the CTE.
- Modern engines inline CTEs; use `MATERIALIZED` only when reusing an expensive subquery multiple times.
- Prevent recursive cycles by tracking a visited path array.

15. Interview Checklist

- [] Understands the three components of a recursive CTE.

- [] Knows how to generate missing date sequences for financial reporting.
- [] Explains **UNION ALL** efficiency over **UNION** in recursion.
- [] Explains CTE inlining vs optimization fences.



CHAPTER 05 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Basic SQL queries and Subqueries.
- Set operations (`UNION` vs `UNION ALL`).
- In older database versions (PostgreSQL < 12), CTEs acted as strict optimization fences. The database computed the CTE on



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 06

ACID Properties & Database Transaction Internals

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

ACID Properties & Database Transaction Internals

1. Why This Matters

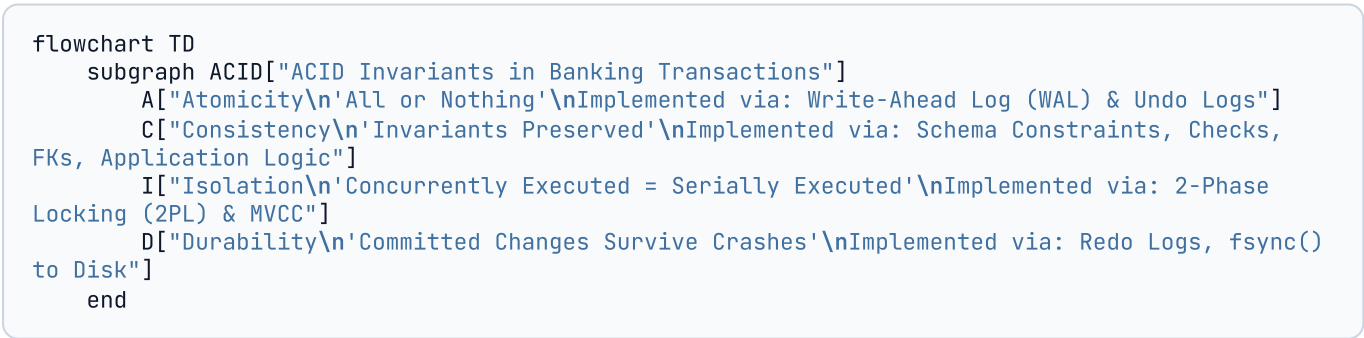
If you remember only one database concept for IDFC FIRST Bank, let it be **ACID**. In banking systems, transactions represent real money. If a server loses power halfway through a ₹10,000 transfer, or two concurrent ATM withdrawals deduct from the same ₹5,000 balance simultaneously, the bank faces direct financial loss and regulatory sanctions from the Reserve Bank of India (RBI). In a 3+ YoE interview, you must explain not just what the acronym stands for, but the **internal physical mechanisms** (Write-Ahead Logging, Two-Phase Locking, Multi-Version Concurrency Control) that guarantee each property.

2. Prerequisites

- Basic relational CRUD operations.
- Understanding of volatile memory (RAM) vs non-volatile storage (SSD/HDD).

3. Concept

The Four Pillars of ACID



1. **Atomicity:** A transaction is an indivisible unit of work. If any statement fails (or the node crashes), all previously executed statements within that transaction are rolled back completely.
2. **Consistency:** A transaction brings the database from one valid state to another, satisfying all declared invariants (`CHECK (balance ≥ 0)` , foreign keys, uniqueness, and business rules like $\Sigma \text{ debits} = \Sigma \text{ credits}$).

3. **Isolation:** Concurrent execution of transactions yields the same state that would be obtained if transactions were executed serially one after the other.
 4. **Durability:** Once a transaction commits, its effects are permanent and will not be lost, even in the event of an immediate total power failure.
-

4. Simple Example: Classic Banking Transfer

```
BEGIN TRANSACTION;

-- Step 1: Deduct from Sender
UPDATE accounts
SET balance = balance - 5000.00
WHERE account_id = 101 AND balance ≥ 5000.00;

-- Step 2: Credit to Receiver
UPDATE accounts
SET balance = balance + 5000.00
WHERE account_id = 103;

-- Step 3: Record in Transaction Ledger
INSERT INTO transactions (transaction_reference, from_account_id, to_account_id, amount,
transaction_type, payment_channel, status)
VALUES ('TXN-MANUAL-001', 101, 103, 5000.00, 'TRANSFER', 'IMPS', 'SUCCESS');

COMMIT;
```

5. Real-World Banking Example: Power Failure Mid-Transfer

Imagine the server CPU dies between Step 1 (debit) and Step 2 (credit):

- **Without ACID Atomicity:** Account 101 has lost ₹5,000, but Account 103 never received it. ₹5,000 has vanished into thin air.
- **With ACID Atomicity (via WAL Undo):** Upon reboot, the database crash recovery process scans the Write-Ahead Log. It sees that transaction `TXN-MANUAL-001` has no corresponding `COMMIT` record. The engine uses the **Undo Log** to restore Account 101's balance to its pre-debit amount before accepting any new incoming client connections.

6. Code: Robust Transaction Handling in Node.js / Express

```

const { Pool } = require('pg');
const pool = new Pool();

async function transferFunds(fromAccountId, toAccountId, amount, idempotencyKey) {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    // 1. Check & Lock Sender Account (Pessimistic Locking)
    const senderRes = await client.query(
      'SELECT balance FROM accounts WHERE account_id = $1 FOR UPDATE',
      [fromAccountId]
    );

    if (senderRes.rows.length === 0) {
      throw new Error('Sender account not found');
    }

    const currentBalance = parseFloat(senderRes.rows[0].balance);
    if (currentBalance < amount) {
      throw new Error('Insufficient funds');
    }

    // 2. Debit Sender
    await client.query(
      'UPDATE accounts SET balance = balance - $1 WHERE account_id = $2',
      [amount, fromAccountId]
    );

    // 3. Credit Receiver
    await client.query(
      'UPDATE accounts SET balance = balance + $1 WHERE account_id = $2',
      [amount, toAccountId]
    );

    // 4. Record Ledger Entry
    await client.query(
      `INSERT INTO transactions
      (transaction_reference, from_account_id, to_account_id, amount, transaction_type,
      payment_channel, status)
      VALUES ($1, $2, $3, $4, 'TRANSFER', 'UPI', 'SUCCESS')`,
      [idempotencyKey, fromAccountId, toAccountId, amount]
    );

    // Commit all changes atomically
    await client.query('COMMIT');
    return { success: true, message: 'Transfer completed successfully' };

  } catch (error) {
    // Rollback on ANY failure
    await client.query('ROLLBACK');
    console.error('Transaction aborted and rolled back:', error.message);
    throw error;
  } finally {
    // Crucial: Always release client back to connection pool
    client.release();
  }
}

```

7. How It Works Internally: Write-Ahead Logging (WAL) & ARIES Recovery

Why Databases Do Not Write Directly to Data Pages on Disk

Modifying a table requires writing to random 8KB/16KB data pages across the disk. Doing random disk I/O on every commit would choke throughput to single-digit transactions per second.

```

flowchart TD
    subgraph CommitFlow ["The Write-Ahead Log (WAL) Flush Pipeline"]
        TX["Transaction Commit Requested"] --> WAL["1. Append Log Record to In-Memory WAL Buffer"]
        WAL --> FSYNC["2. Issue synchronous fsync() to WAL Disk (Sequential I/O - Fast!)"]
        FSYNC --> ACK["3. Send Success ACK to Client"]
        ACK --> BG["4. Later: Background DB Checkpointer flushes dirty data pages to disk"]
    end
  
```

The WAL Invariant

KEY ENGINEERING INSIGHT

No modified data page (dirty page) is EVER written to non-volatile storage until the corresponding log records describing the change have been flushed to disk (`fsync`).

Crash Recovery: The ARIES Protocol

When a database reboots after an abrupt power loss, it performs 3 phases: 1. **Analysis Phase**: Identifies all dirty pages in buffer pool and all active (uncommitted) transactions at the time of the crash. 2. **Redo Phase**: Replays all log records up to the crash point, restoring the database to the exact state it was in before the failure (repeating history). 3. **Undo Phase**: Reverses the changes of all transactions that were active but had not committed when the crash occurred, restoring complete atomicity.

8. Common Mistakes

1. **Forgetting** `client.release()` in `finally` : If an error occurs and the connection is not released, database connection pools exhaust rapidly, resulting in cascading server outages.
2. **Long-Running Business Logic Inside Transactions**: Calling an external HTTP API (like an SMS gateway or partner payment switch) *inside* an open SQL transaction holds locks for seconds, causing deadlocks and blocking unrelated users. **Rule: Never make external network I/O calls inside a database transaction.**
3. **Assuming** `COMMIT` is Asynchronous by Default: Disabling `fsync` (`synchronous_commit = off`) improves write speed dramatically, but breaks Durability: committed transactions can be lost upon power failure.

9. Performance / Complexity Matrix

Property	Implementation Mechanism	Performance Overhead
Atomicity	Undo logs / WAL append	Minimal sequential write overhead
Consistency	Constraints & Foreign Key checks	$O(\log N)$ index lookups on insert/update
Isolation	Locking / MVCC snapshots	Concurrency contention, lock waits
Durability	Sequential <code>fsync()</code> to WAL	Constrained by disk IOPs (mitigated by group commit)

● Easy Question: ACID Properties Defined

Q: What does ACID stand for? Define each property with a single authoritative sentence and its concrete database enforcement mechanism.

Complete Answer:

- Atomicity ("All or Nothing"):** - *Definition:* Either all operations within a transaction succeed and are committed, or the entire transaction is aborted and any partial modifications are completely rolled back. - *Mechanism:* Enforced via Undo Logs (MySQL InnoDB) or MVCC transaction status bitmaps and Write-Ahead Log (PostgreSQL).
- Consistency ("Valid State to Valid State"):** - *Definition:* A transaction can only transition the database from one valid state to another, preserving all declared schema constraints, foreign keys, unique indexes, and banking financial invariants (e.g., $\sum \text{debits} = \sum \text{credits}$). - *Mechanism:* Enforced via database engine constraint validators (Primary Keys, Foreign Keys, `CHECK` constraints, triggers) and application double-entry balance rules.
- Isolation ("Independent Execution"):** - *Definition:* Concurrent transactions execute without interfering with one another; the intermediate, uncommitted modifications of one transaction remain invisible to other concurrent transactions. - *Mechanism:* Enforced via Multi-Version Concurrency Control (MVCC) snapshots, row-level locks, and table intention locks.
- Durability ("Permanent Survival"):** - *Definition:* Once a transaction commits and the application receives a success acknowledgment, its changes survive all subsequent hardware failures, operating system crashes, or power outages. - *Mechanism:* Enforced by sequentially flushing log records to non-volatile disk storage via the synchronous OS `fsync()` system call before acknowledging the commit (Write-Ahead Logging).

● Medium Question: How Write-Ahead Logging (WAL) Guarantees Durability

Q: Explain how Write-Ahead Logging (WAL) allows a database to guarantee Durability without immediately flushing entire data pages to disk. What is the ARIES recovery protocol?

Complete Answer:

- The Performance Problem of Direct Page Flushing:** - Database tables are stored as fixed-size pages (8 KB in PostgreSQL, 16 KB in MySQL InnoDB). - If an update changes a single 4-byte integer in a customer record, flushing the entire 8 KB page to disk requires a **random I/O write**. - With hundreds of concurrent transactions touching scattered accounts, random disk writes would saturate storage controller IOPS immediately, cratering throughput to double digits.
- The WAL Solution:** - When data is modified, changes are applied to the in-memory **Buffer Pool** in RAM (marking the page as a "dirty page"). - A compact, append-only log record describing the delta (e.g., `"Txn 402: Account 101`

balance decremented from 5000 to 4900") is written to the **WAL Buffer**. - On **COMMIT**, only the WAL buffer is flushed sequentially to disk via **fsync()**. Because sequential write throughput is 100x to 1000x faster than random I/O, commit latency is negligible (sub-millisecond). - Dirty pages remain in RAM and are lazily flushed in large, sorted batches to disk by background checkpoint threads. 3. **Crash Recovery via the ARIES Protocol**: If power is cut while dirty pages are still in RAM, the engine boots into ARIES recovery: - **Phase 1 (Analysis)**: Scans the WAL forward from the last checkpoint to identify active transactions that never committed and all dirty pages. - **Phase 2 (Redo / "Repeating History")**: Scans the WAL forward and reapplies all logged operations (including those of uncommitted transactions) to restore the buffer pool to the exact state at the microsecond of the crash. - **Phase 3 (Undo)**: Scans the WAL backward and rolls back the changes of all transactions that were active without a committed record, restoring strict atomicity.

● Hard Question: Concurrent Balance Deduction Race Condition

Q: If two concurrent transactions execute `BEGIN ... UPDATE accounts SET balance = balance - 100 WHERE account_id = 1 ... COMMIT` at the default **READ COMMITTED** isolation level, can a race condition allow the balance to go negative? Walk through the exact row-level locking behavior.

Complete Answer: - The Detailed Row-Locking Sequence: 1. Transaction 1 begins and issues `UPDATE ... WHERE account_id = 1`. The engine acquires an **exclusive row-level write lock (X-lock)** on account 1. 2. Transaction 2 begins concurrently and issues `UPDATE ... WHERE account_id = 1`. Because an X-lock is already held by Txn 1, Txn 2 enters a **lock wait state** and blocks. 3. Txn 1 completes its subtraction (e.g., initial balance 100 becomes 0) and issues **COMMIT**. Txn 1 releases the X-lock. 4. Txn 2 immediately wakes up. At **READ COMMITTED** isolation, the database engine uses **Read Committed Row Re-evaluation**: - It fetches the newly committed version of row 1 (where balance is now 0). - It re-evaluates the query: `UPDATE accounts SET balance = balance - 100 WHERE account_id = 1`. - Because the **WHERE** clause only checked `account_id = 1` (which is still true!), it computes `balance = 0 - 100 = -100`! - Txn 2 commits. The balance is now **-100 INR!** - **The Production Remediation:** To guarantee positive balances under high concurrency without requiring heavy **SERIALIZABLE** isolation: 1. **Atomic Guard Condition in WHERE Clause:** `sql UPDATE accounts SET balance = balance - 100 WHERE account_id = 1 AND balance ≥ 100;` When Txn 2 wakes up and re-evaluates the **WHERE** clause, `balance ≥ 100` evaluates to **FALSE** (`0 ≥ 100` is False). The row is not updated (`ROW_COUNT() = 0`). The application detects zero rows updated and returns an **"Insufficient Funds"** error. 2. **Database Schema Invariant:** Always enforce a defensive table constraint: `sql ALTER TABLE accounts ADD CONSTRAINT chk_positive_balance CHECK (balance ≥ 0);` Even if buggy code forgets the **WHERE** clause guard, the engine aborts the transaction before committing.

11. Follow-up Questions from Interviewer with Complete Answers

🎤 Follow-up 1

Interviewer: "What is Group Commit in PostgreSQL/MySQL InnoDB, and how does it prevent disk I/O bottlenecks during peak transaction bursts?"

Complete Answer: 1. **The Disk Sync Bottleneck:** - Every individual transaction **COMMIT** requires calling the OS system call **fsync()** to ensure the WAL buffer is physically flushed from OS disk cache to the non-volatile drive. - An **fsync()** call blocks the thread until the drive controller sends hardware ACK (1 to 5 ms on mechanical drives; 0.1 to 0.3 ms on enterprise NVMe). - If 10,000 requests arrive per second and each demands an immediate, dedicated **fsync()**, the storage controller saturates, queueing commits and stalling the application. 2. **The Group Commit Optimization:** - When transaction T_1 requests a commit, the engine designates T_1 's thread as the **group leader**. - While the group leader prepares the write, dozens of other transactions ($T_2, T_3, \dots, T_{50}$) also finish and request commits. - Instead of issuing 50 separate **fsync()** calls, the engine appends all 50 commit records into a single contiguous disk buffer. - The group leader executes a **single batched fsync() call** that commits all 50 transactions simultaneously! - The group leader wakes up the follower threads and all 50 transactions receive their commit acknowledgments in parallel. - **Throughput Multiplier:** Scales transaction commit throughput by $10\times$ to $50\times$ during peak traffic spikes (e.g., Diwali or month-end payroll processing).

Follow-up 2

Interviewer: "What is the difference between a Local Transaction and a Distributed Transaction across two microservices? Why can't we use standard ACID transactions across microservices, and how does the Saga Pattern solve it?"

Complete Answer: 1. **Local vs Distributed Transactions:** - **Local Transaction:** Operates within a single database instance. The database engine controls all locks, buffers, and WAL locally, guaranteeing strict ACID. - **Distributed Transaction:** Spans two or more separate databases (e.g., Core Banking Account Service DB and Fraud Detection Service DB). 2. **Why ACID Cannot Scale Across Microservices (The 2PC Problem):** - The traditional solution was **Two-Phase Commit (2PC)** coordinated by a transaction manager. - **Failure Mode:** 2PC is a **synchronous blocking protocol**. If the coordinator or any participating service crashes during the commit phase, all participating databases hold row locks indefinitely. - **Microservice Antipattern:** Violates service autonomy. If Service B is slow, Service A's database connection pools freeze. 3. **The Modern FinTech Solution: The Saga Pattern:** - Breaks the distributed transaction into a sequence of **local ACID transactions** coordinated via asynchronous messaging (Apache Kafka): 1. Account Service locally debits ₹5,000 from customer account (Local ACID). Publishes **FundsDebited** event. 2. NPCI Switch Gateway receives event, transmits to beneficiary bank (Local ACID). - **Handling Failures (Compensating Transactions):** If the beneficiary bank rejects the transfer, the Saga orchestrator issues a **compensating transaction** to Account Service: **Credit ₹5,000 back to customer account with reason: 'Transfer Reversed'**. - The system achieves **Eventual Consistency** without holding distributed locks.

13. Practical Exercise

Verify SQLite transactional rollback on **banking.db**:

```
sqlite3 banking.db
```

```

BEGIN TRANSACTION;
-- Simulate a debit
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 101;
-- Inspect balance (it is decremented)
SELECT account_id, balance FROM accounts WHERE account_id = 101;
-- Abort transaction
ROLLBACK;
-- Verify balance has been restored!
SELECT account_id, balance FROM accounts WHERE account_id = 101;

```

14. Quick Revision

- Atomicity: All or nothing, managed by WAL undo logs.
- Consistency: Business rules & constraints preserved.
- Isolation: Concurrent transactions don't interfere.
- Durability: Committed transactions survive crashes via `fsync` to sequential WAL.
- ARIES recovery: Analysis → Redo → Undo.
- Never place external HTTP network calls inside open database transactions.

15. Interview Checklist

- [] Can articulate the physical mechanism for each of the 4 ACID letters.
- [] Understands why WAL uses sequential I/O while data pages use random I/O.
- [] Knows the 3 phases of ARIES recovery.
- [] Explains why connection pool client release must always reside in a `finally` block.

CHAPTER 06 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Basic relational CRUD operations.
- Understanding of volatile memory (RAM) vs non-volatile storage (SSD/HDD).
- Without ACID Atomicity: Account 101 has lost ₹5,000, but Account 103 never received it. ₹5,000 has vanished into thin air

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 07

Database Isolation Levels, Read Phenomena & MVCC

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Database Isolation Levels, Read Phenomena & MVCC

1. Why This Matters

In banking applications, isolation level configuration is the difference between an engine processing 20,000 transactions/sec safely and an engine losing money to race conditions or deadlocking into complete paralysis. In technical interviews for IDFC FIRST Bank, interviewers will ask you to define the three classical read phenomena (Dirty Read, Non-Repeatable Read, Phantom Read) and explain how modern relational engines (PostgreSQL, MySQL InnoDB) use **Multi-Version Concurrency Control (MVCC)** rather than pure lock-based isolation.

2. Prerequisites

- ACID properties (specifically Isolation).
- Basic transactional syntax (`BEGIN` , `COMMIT` , `ROLLBACK`).

3. Concept

The 3 Classical Read Phenomena

- Dirty Read:** Transaction T_1 modifies a row. Transaction T_2 reads the uncommitted row. If T_1 subsequently aborts (`ROLLBACK`), T_2 has read data that technically never existed in the database.
- Non-Repeatable Read (Fuzzy Read):** Transaction T_1 reads a row. Transaction T_2 modifies or deletes that row and commits. If T_1 re-reads the row, it sees a *different value* or finds the row gone.
- Phantom Read:** Transaction T_1 executes a range query (e.g., `WHERE balance > 50000`). Transaction T_2 inserts a *new row* matching that predicate and commits. When T_1 executes the exact same query again, a "phantom" new row appears.



4. Simple Example: Dirty Read Danger

Time	Transaction 1 (Debiting Account A)	Transaction 2 (Loan Eligibility Checker)
t1	BEGIN;	
t2	UPDATE accounts SET balance = 5000000 WHERE account_id = 101; -- Temporary ledger stage	
t3		BEGIN;
t4		SELECT balance FROM accounts WHERE account_id = 101;
		-- Reads uncommitted 5,000,000! Approves loan!
t5	ROLLBACK; -- Oops! Invalid wire transfer!	
t6		COMMIT;

5. Real-World Banking Example: The Lost Update Anomaly

Suppose Account 101 has a balance of ₹10,000. Two ATM withdrawals of ₹6,000 are attempted at the exact same millisecond:

Transaction A (ATM 1)	Transaction B (ATM 2)
1. SELECT balance (reads 10,000)	
2.	1. SELECT balance (reads 10,000)
3. App calculates: 10,000 - 6,000 = 4,000	
4.	2. App calculates: 10,000 - 6,000 = 4,000
5. UPDATE balance = 4,000	
6. COMMIT	
7.	3. UPDATE balance = 4,000
8.	4. COMMIT

- **Result:** Both customers received ₹6,000 cash (₹12,000 total dispensed), but the balance in the database is ₹4,000 instead of ₹-2,000 or rejecting the second withdrawal!
- **Why it happened:** In standard `READ COMMITTED`, readers do not block writers, and writers do not block readers.
- **The Fix:** Pessimistic locking (`SELECT ... FOR UPDATE`) or atomic SQL decrement (`UPDATE accounts SET balance = balance - 6000 WHERE balance ≥ 6000`).

6. Code: Configuring Isolation Levels in PostgreSQL / Node.js

```
const { Pool } = require('pg');
const pool = new Pool();

async function runSerializableAudit() {
  const client = await pool.connect();
  try {
    // Set transaction isolation to SERIALIZABLE
    await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');

    const highNetWorthTally = await client.query(
      'SELECT COUNT(*), SUM(balance) FROM accounts WHERE balance ≥ 100000'
    );

    // Perform reporting calculations...

    await client.query('COMMIT');
  } catch (err) {
    // Under SERIALIZABLE, concurrent conflicts throw serialization_failure (error code 40001)
    if (err.code === '40001') {
      console.warn('Serialization failure detected! Retry transaction.');
```

7. How It Works Internally: Multi-Version Concurrency Control (MVCC)

Modern relational engines avoid locking tables for read queries by maintaining multiple physical versions of each row.

PostgreSQL Row Tuple Headers: `xmin` and `xmax`

Every physical row tuple contains hidden system attributes: - `xmin`: The transaction ID (XID) that inserted this row version. - `xmax`: The transaction ID (XID) that deleted or updated (superseded) this row version (0 if active).

```
flowchart LR
  subgraph RowVersions["Tuple Evolution under MVCC"]
    R1["Row v1: Balance 10,000\nxmin: 100, xmax: 105 (Superseded)"]
    R2["Row v2: Balance 4,000\nxmin: 105, xmax: 0 (Live)"]
  end
  R1 --> R2
```

Snapshot Isolation

- When Transaction T runs a query, the engine captures a **Read Snapshot** containing:
- Active transaction IDs that have not yet committed.
- The highest committed transaction ID.

- **Rule:** Readers never block writers, and writers never block readers! When a write occurs, a new row version is appended, and the old version remains visible to concurrent snapshots.
- **Vacuuming:** The PostgreSQL `VACUUM` background daemon cleans up obsolete row versions (dead tuples) once no active transaction snapshot needs them.

8. Common Mistakes

1. **Assuming `READ COMMITTED` Prevents Lost Updates:** `READ COMMITTED` ensures you only read committed data, but it does **not** prevent another transaction from overwriting your change if you read, calculate in application memory, and write back.
2. **Not Handling Serialization Failures (Error 40001):** When using `SERIALIZABLE` isolation, the database will abort transactions that exhibit non-serializable interleaving. **Any application using Serializable isolation MUST implement exponential backoff retry loops!**
3. **Leaving Transactions Open in Repeatable Read:** In PostgreSQL, long-running transactions at `REPEATABLE READ` prevent `VACUUM` from removing dead tuples, leading to severe **table bloat** and memory exhaustion.

9. Performance / Complexity Matrix

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Concurrency Throughput
Read Uncommitted	Possible	Possible	Possible	Maximum
Read Committed	Prevented	Possible	Possible	Very High (Industry Standard)
Repeatable Read	Prevented	Prevented	Prevented in Postgres	High
Serializable	Prevented	Prevented	Prevented	Lowest (High abort/retry rate)

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is a Dirty Read and which isolation level is the minimum required to prevent it?

Medium

- **Q:** Explain the difference between a Non-Repeatable Read and a Phantom Read. Give a concrete banking example of each.

Hard

- **Q:** How does Serializable Snapshot Isolation (SSI) in PostgreSQL detect serialization anomalies without taking coarse table-level read locks?

11. Follow-up Questions from Interviewer

- "Why does MySQL InnoDB default to **REPEATABLE READ** while PostgreSQL and Oracle default to **READ COMMITTED**?" (Answer: Historically, MySQL replication relied on statement-based binary logging (**binlog**), which required Repeatable Read with next-key locks to prevent replica state drift).
- "What is a Write Skew anomaly, and why can it occur under Repeatable Read but not under Serializable?"

12. Model Answer: Write Skew Under Repeatable Read

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "Can you explain the Write Skew anomaly and why Repeatable Read cannot prevent it?"

Model Answer: "The **Write Skew** anomaly occurs when two concurrent transactions read overlapping data sets, make decisions based on what they read, and write to *disjoint* rows, violating a global business constraint.

Banking Example: Combined Minimum Balance Constraint: – Rule: A customer can hold Account A and Account B, provided their *combined* balance remains $\geq ₹1,000$. – Currently: Account A has ₹800, Account B has ₹800 (Total = ₹1,600). – Transaction 1 attempts to withdraw ₹700 from Account A. It reads both accounts (sum = ₹1,600 $\geq$ 1,000), approves the debit, and updates Account A to ₹100. – Concurrently, Transaction 2 attempts to withdraw ₹700 from Account B. Under **REPEATABLE READ**, its snapshot sees Account A = ₹800 and Account B = ₹800 (sum = ₹1,600), approves the debit, and updates Account B to ₹100. – Both transactions commit because they modified *different* rows (no row-level lock conflict!). – Final state: Account A = ₹100, Account B = ₹100 (Total = ₹200). The global invariant is violated!

To prevent Write Skew, the system must either use **SERIALIZABLE** isolation (where dependency tracking detects the cycle) or explicitly lock both rows using **SELECT ... FOR UPDATE**."

13. Practical Exercise

Inspect isolation level support in SQLite or PostgreSQL:

```
sqlite3 banking.db
```

```
-- SQLite supports READ UNCOMMITTED and SERIALIZABLE
PRAGMA read_uncommitted = 0; -- Default: Full Serializable transaction isolation
```

14. Quick Revision

- Dirty Read: Reading uncommitted data.
- Non-Repeatable Read: Row value changes between two reads in the same transaction.
- Phantom Read: New rows appear matching a range predicate.
- ANSI levels: Read Uncommitted → Read Committed → Repeatable Read → Serializable.
- MVCC ensures readers never block writers and writers never block readers.
- Write Skew is possible in Repeatable Read; requires Serializable or explicit locking.

15. Interview Checklist

- ☐ Clearly distinguishes Non-Repeatable Read from Phantom Read.
- ☐ Explains `xmin` / `xmax` and MVCC snapshots.
- ☐ Understands the Lost Update problem and explains 2 ways to prevent it.
- ☐ Can articulate the Write Skew scenario and why Repeatable Read fails to stop it.

 **CHAPTER 07 SUMMARY & REVISION CHECKLIST**

 **WHAT TO REMEMBER**

- ACID properties (specifically Isolation).
- Basic transactional syntax (``BEGIN``, ``COMMIT``, ``ROLLBACK``).
- Result: Both customers received ₹6,000 cash (₹12,000 total dispensed), but the balance in the database is ₹4,000 instead

 **CANDIDATE SELF-CHECK**

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 08

Database Locking, Pessimistic vs Optimistic Concurrency & Deadlocks

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Database Locking, Pessimistic vs Optimistic Concurrency & Deadlocks

1. Why This Matters

In banking transactions at IDFC FIRST Bank, multiple payment channels (UPI, ATM, Netbanking, POS terminals) may concurrently attempt to debit from or credit to the exact same customer account. If concurrency control is misconfigured:

- A customer with ₹1,000 can withdraw ₹1,000 simultaneously from two devices (double spending).
- Two cross-transfers (A → B and B → A) can lock each other's accounts, freezing the database into a **deadlock**.

In your interview, you will be expected to contrast **Pessimistic Locking** vs **Optimistic Locking**, explain the exact SQL syntax for each, and explain how databases detect and resolve deadlocks.

2. Prerequisites

- ACID properties (specifically Isolation and Atomicity).
- Basic transactional syntax (`BEGIN` , `COMMIT` , `ROLLBACK`).

3. Concept

Lock Modes

1. **Shared Lock (S-Lock / Read Lock)**: Multiple transactions can hold shared locks on the same row concurrently for reading (`SELECT ... FOR SHARE`). Writers are blocked until all S-locks are released.
2. **Exclusive Lock (X-Lock / Write Lock)**: Only one transaction can hold an X-lock on a row (`SELECT ... FOR UPDATE` or `UPDATE` / `DELETE`). All other readers and writers are blocked.

flowchart TD

subgraph LockingStrategies["Concurrency Control Paradigms"]

direction TB

P["Pessimistic Locking\n'Assume conflict will happen'\nAcquire Exclusive Lock at read time: SELECT ... FOR UPDATE\nGuarantees safety; reduces concurrency."]

O["Optimistic Locking\n'Assume conflict is rare'\nRead without locks; verify version column on write: WHERE id = 1 AND version = 5\nHigh concurrency; requires application retry loop."]

end

4. Simple Example: Pessimistic vs Optimistic Update

-- 1. Pessimistic Locking (SELECT ... FOR UPDATE)

BEGIN;

SELECT balance FROM accounts WHERE account_id = 101 FOR UPDATE;

-- Balance is now locked; any concurrent transaction attempting to read or write this row BLOCKS!

UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;

COMMIT;

-- 2. Optimistic Locking (Version Column)

-- Step A: Application reads without locking:

SELECT balance, version FROM accounts WHERE account_id = 101; -- Suppose balance = 1000, version = 1

-- Step B: Application updates only if version has NOT changed:

UPDATE accounts

SET balance = balance - 500, version = version + 1

WHERE account_id = 101 AND version = 1;

-- If rows affected = 0, another transaction updated first! Abort and retry.

5. Real-World Banking Example: The Cross-Transfer Deadlock

Customer 1 transfers ₹5,000 to Customer 2, while Customer 2 concurrently transfers ₹3,000 to Customer 1:

Transaction 1 (User 1 → User 2)

Transaction 2 (User 2 → User 1)

t1: Locks Account 1 (X-Lock acquired)

t2:

Locks Account 2 (X-Lock acquired)

t3: Requests Lock on Account 2 (WAITS...)

t4:

Requests Lock on Account 1 (WAITS...)

DEADLOCK! Neither transaction can proceed!

flowchart LR

T1["Transaction 1\n(Holds Acc 1)"] -->|Waiting for Acc 2| T2["Transaction 2\n(Holds Acc 2)"]

T2 -->|Waiting for Acc 1| T1

6. Code: Production-Grade Optimistic Locking with Retry in Node.js

```

async function debitAccountOptimistic(accountId, debitAmount, maxRetries = 3) {
  for (let attempt = 1; attempt ≤ maxRetries; attempt++) {
    const client = await pool.connect();
    try {
      // 1. Read current state without locking
      const res = await client.query(
        `SELECT balance, version FROM accounts WHERE account_id = $1`,
        [accountId]
      );

      if (res.rows.length === 0) throw new Error('Account not found');

      const currentBalance = parseFloat(res.rows[0].balance);
      const currentVersion = parseInt(res.rows[0].version);

      if (currentBalance < debitAmount) {
        throw new Error('Insufficient funds');
      }

      // 2. Attempt update conditioned on version match
      const updateRes = await client.query(
        `UPDATE accounts
        SET balance = balance - $1, version = version + 1
        WHERE account_id = $2 AND version = $3`,
        [debitAmount, accountId, currentVersion]
      );

      // If affected rows = 1, update succeeded!
      if (updateRes.rowCount === 1) {
        return { success: true, newBalance: currentBalance - debitAmount };
      }

      // Version mismatch: another transaction committed first
      console.warn(`Version conflict on account ${accountId}. Retrying
        (${attempt}/${maxRetries})...`);
      await new Promise((r) => setTimeout(r, Math.random() * 50 * attempt)); // Jittered
      backoff

    } finally {
      client.release();
    }
  }
  throw new Error('Transaction failed after maximum retries due to high concurrency');
}

```

7. How It Works Internally: Deadlock Detection & Wait-For Graphs

The Lock Manager & Wait-For Graph (WFG)

The database engine maintains a directed graph in shared memory: – **Nodes:** Active transactions. – **Edges:** Directed from transaction T_A to transaction T_B if T_A is waiting for a lock held by T_B.

Deadlock Detection Daemon

1. A background thread runs at regular intervals (e.g., `deadlock_timeout = 1s` in PostgreSQL).
2. It executes **Tarjan's strongly connected components algorithm** or **Depth-First Search (DFS)** to detect cycles in the Wait-For Graph.

3. If a cycle is detected, the engine breaks the deadlock by selecting a **victim transaction** (typically the transaction with the fewest modified rows or lowest cost to abort), terminates it with error `40P01 ( deadlock_detected )`, and rolls it back, allowing the other transaction to complete.

8. Common Mistakes

1. **Inconsistent Lock Ordering Across Services:** `text Service A locks: Account A, then Account B`
`Service B locks: Account B, then Account A` **Golden Rule:** Always acquire locks in a globally deterministic order (e.g., sort account IDs ascending before locking: `account_id_min` first, then `account_id_max`). This makes cycles in the Wait-For Graph mathematically impossible!
2. **Locking Rows During Long Third-Party API Calls:** If your code locks a bank account row, calls an external SMS gateway or third-party bank verification API (which hangs for 5 seconds), all other operations on that account queue up, exhausting pool connections.
3. **Using `SELECT ... FOR UPDATE` Without `SKIP LOCKED` or `NOWAIT` for Task Queues:** In message processing tables, concurrent workers trying to fetch tasks with `FOR UPDATE` will pile up behind worker 1. Use `SELECT ... FOR UPDATE SKIP LOCKED` to allow workers to grab distinct rows concurrently.

9. Performance / Complexity Matrix

Strategy	Read Overhead	Write Overhead	Best Used When
Pessimistic (<code>FOR UPDATE</code>)	High (Acquires exclusive lock)	High (Contention blocks readers)	High contention, high probability of collision (e.g., flash sales, single balance updates).
Optimistic (Version Key)	Zero (Standard MVCC read)	Low (Single conditional write)	Low-to-moderate contention, mostly reads.
Deterministic Ordering	Zero	Zero	Eliminates deadlocks completely in multi-entity transfers.

● Easy Question: Shared Lock (S) vs Exclusive Lock (X)

Q: What is the fundamental difference between a Shared Lock and an Exclusive Lock? Provide their compatibility matrix and SQL examples.

Complete Answer: 1. **Lock Definitions:** – **Shared Lock (S-Lock / Read Lock):** Acquired when reading data without mutating it. Multiple transactions can hold concurrent shared locks on the same row. – **Exclusive Lock (X-Lock / Write Lock):** Acquired when modifying data (`INSERT` , `UPDATE` , `DELETE`) or explicitly reserving a row (`FOR UPDATE`). Only ONE transaction can hold an exclusive lock on a row at any given moment; all other readers and writers are blocked. 2. **Lock Compatibility Matrix:** | Requested \ Held | None | Shared (S) | Exclusive (X) | | :--- | :---: | :---: | :---: | | **Shared (S)** | Granted | Granted

(Compatible) | Blocked (Wait) | | **Exclusive (X)** | Granted | Blocked (Wait) | Blocked (Wait) | 3. **Explicit SQL Syntax:** ``sql -- Acquires Shared Lock (allows other transactions to read, prevents writes) SELECT * FROM accounts WHERE account_id = 101 FOR SHARE;

-- Acquires Exclusive Lock (blocks all other transactions from reading with lock or writing) SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE;

4. ****Intent Locks at Table Level:**** - When an engine locks a row, it first acquires an ****Intent Shared (IS)**** or ****Intent Exclusive (IX)**** lock on the parent table. This allows a table-level operation (like ALTER TABLE or LOCK TABLE) to instantly check whether any child rows are locked in O(1) time without scanning millions of individual row locks.

● Medium Question: Pessimistic vs Optimistic Locking in Banking Architecture

Q: Compare Pessimistic Locking vs Optimistic Locking. In what banking scenarios would you choose one over the other?

Complete Answer: 1. **Mechanics Comparison:** - **Pessimistic Locking (SELECT ... FOR UPDATE):** - Assumes conflicts *will* happen. The transaction locks the row before reading its state and holds the exclusive lock until **COMMIT** or **ROLLBACK**. - *Advantage:* Guaranteed consistency; zero failed writes or rollback retries. - *Disadvantage:* High lock contention; other transactions block in connection pools, reducing system throughput. - **Optimistic Locking (Version Column):** - Assumes conflicts are *rare*. Reads without locks via MVCC. When writing, checks if the version has changed: `sql UPDATE accounts SET balance = balance - 500, version = version + 1 WHERE account_id = :id AND version = :read_version;` - *Advantage:* Zero database locks; high throughput; ideal for distributed reads. - *Disadvantage:* If another transaction committed first, rows affected = 0. The application must roll back and retry. 2. **Banking Architectural Decision Matrix:** - **Use Pessimistic Locking for:** Core Ledger Balance Deductions and UPI Fund Transfers. When 100 concurrent payments hit a merchant account, optimistic retry loops will burn CPU and network cycles repeatedly failing. A pessimistic queue on the ledger row serializes payments cleanly. - **Use Optimistic Locking for:** Customer Profile Updates, KYC Document Verification, and Loan Application Workflows. Here, users take minutes to fill forms; holding a database lock during user think-time would exhaust DB connection pools.

● Hard Question: Deadlock-Free Fund Transfers via Deterministic Lock Ordering

Q: If User A transfers money to User B, and User B simultaneously transfers money to User A, how do you mathematically guarantee that no database deadlock will ever occur?

Complete Answer: - **Root Cause of Transfer Deadlocks:** - Txn 1 (A → B): Locks Account A, then attempts to lock Account B. - Txn 2 (B → A): Locks Account B, then attempts to lock Account A. - Both transactions now wait for each other, creating a circular cycle in the Wait-For Graph (Deadlock **40P01**). - **The Solution: Deterministic Resource Ordering:** - Coffman's condition for deadlocks requires **Circular Wait**. To eliminate circular wait mathematically, enforce a strict global partial order on lock acquisitions: `first_id = min(acc_from, acc_to)` `second_id = max(acc_from, acc_to)` - Both transactions are

forced to acquire locks in ascending order of `account_id`. - Txn 1 and Txn 2 will both contend for `min(A, B)` first. The winner proceeds to lock `max(A, B)`, while the loser waits before acquiring any lock. A circular cycle is mathematically impossible!

```
import psycopg2

def execute_deadlock_free_transfer(conn, from_acc: int, to_acc: int, amount: float):
    """Executes a dual-account ledger transfer guaranteed to never deadlock."""
    # Step 1: Enforce deterministic ascending lock order
    first_lock_id = min(from_acc, to_acc)
    second_lock_id = max(from_acc, to_acc)

    with conn.cursor() as cur:
        # Step 2: Acquire row locks deterministically
        cur.execute(
            "SELECT account_id, balance FROM accounts WHERE account_id = %s FOR UPDATE;",
            (first_lock_id,)
        )
        cur.execute(
            "SELECT account_id, balance FROM accounts WHERE account_id = %s FOR UPDATE;",
            (second_lock_id,)
        )

        # Step 3: Verify source balance
        cur.execute("SELECT balance FROM accounts WHERE account_id = %s;", (from_acc,))
        current_bal = cur.fetchone()[0]
        if current_bal < amount:
            conn.rollback()
            raise ValueError("Insufficient Funds")

        # Step 4: Perform atomic balance adjustments
        cur.execute("UPDATE accounts SET balance = balance - %s WHERE account_id = %s;",
            (amount, from_acc))
        cur.execute("UPDATE accounts SET balance = balance + %s WHERE account_id = %s;",
            (amount, to_acc))

        # Step 5: Record double-entry ledger audit row
        cur.execute(
            "INSERT INTO ledger_entries (from_acc, to_acc, amount, status) VALUES (%s, %s, %s, %s);",
            (from_acc, to_acc, amount, 'SUCCESS')
        )
    conn.commit()
```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "What is the difference between `SELECT ... FOR UPDATE NOWAIT` and `SELECT ... FOR UPDATE SKIP LOCKED`? When would you use each in a high-throughput banking system?"

Complete Answer: 1. **NOWAIT (Fail Fast):** - If the requested row is currently locked by another transaction, `NOWAIT` immediately aborts the query and raises an error (`55P03: could not obtain lock on row in relation...`). - **Banking Use Case:** High-frequency customer card swiping or ATM withdrawals. Rather than letting the payment gateway connection hang for 10 seconds waiting on a locked row (which exhausts application thread pools), the system fails immediately with a clean error (`"Concurrent transaction in progress; please retry"`). 2. **SKIP LOCKED (High-Concurrency Work**

Queues): - If the requested row is locked, `SKIP LOCKED` does not wait and does not throw an error: it silently skips the locked row and returns the first available *unlocked* row matching the `WHERE` condition.

- **Banking Use Case:** Distributed asynchronous payment dispatchers (e.g. processing 10,000 pending NEFT/RTGS settlement batches). ``sql -- 10 worker microservices run this concurrently with 0% lock contention: BEGIN; SELECT payment_id, amount, beneficiary_ifsc FROM pending_payments WHERE status = 'PENDING' ORDER BY priority DESC, created_at ASC LIMIT 1 FOR UPDATE SKIP LOCKED;

```
-- Process payment with external clearing house...
UPDATE pending_payments SET status = 'PROCESSING' WHERE payment_id = :id;
COMMIT;
```

- Each worker grabs a distinct, non-overlapping batch with zero contention and zero deadlocks.

Follow-up 2

Interviewer: "How does Two-Phase Locking (2PL) differ from Two-Phase Commit (2PC)? Why are their names so similar but their architectural purposes completely different?"

Complete Answer: - **The Core Distinction:** - **2PL (Two-Phase Locking):** A **single-node concurrency control protocol** ensuring serializability across concurrent transactions within a single database engine.

- **2PC (Two-Phase Commit):** A **distributed consensus protocol** ensuring atomicity across multiple distinct network nodes or heterogeneous databases (Distributed Transactions). - **2PL Breakdown:** -

Growing Phase: A transaction may acquire locks, but cannot release any lock. - **Shrinking Phase:** A transaction may release locks, but cannot acquire any new locks. - **Strict 2PL (SS2PL):** Prevents cascading aborts by holding all exclusive locks until `COMMIT` / `ROLLBACK`. All major RDBMS (PostgreSQL, MySQL InnoDB, Oracle) implement Strict 2PL. - **2PC Breakdown:** -

Prepare Phase (Voting): The coordinator asks all participating resource managers (e.g., Bank DB and Fraud Service DB): "Can you commit txn X?" Participants execute local operations, write prepare records to WAL, and reply `VOTE_COMMIT` or `VOTE_ABORT`. - **Commit Phase:** If all vote yes, coordinator writes `COMMIT` to disk and commands participants to commit. If any votes no, coordinator broadcasts `ABORT`. - **Architectural Trade-off in FinTech:**

13. Practical Exercise

Inspect the schema version column on `banking.db` :

```
sqlite3 banking.db
```

```
-- Notice that the accounts table created in schema.sql already contains a version column!
SELECT account_id, account_number, balance, version FROM accounts;

-- Test an optimistic update:
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE account_id = 101 AND version = 1;

-- Verify row count was 1 and version is now 2:
SELECT account_id, balance, version FROM accounts WHERE account_id = 101;
```

14. Quick Revision

- Shared lock (Read) allows multiple readers; Exclusive lock (Write) allows only 1 transaction.
- Pessimistic: locks early (`FOR UPDATE`); Optimistic: checks version on commit (`WHERE version = v`).
- Deadlocks require 4 Coffman conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.
- Sort entity IDs before locking to guarantee deadlock-free execution.
- `SKIP LOCKED` enables scalable distributed worker queues without lock contention.

15. Interview Checklist

- [] Clearly articulates the tradeoffs between Pessimistic and Optimistic locking.
- [] Writes the exact SQL for an optimistic version check.
- [] Explains deterministic lock ordering to eliminate transfer deadlocks.
- [] Explains `SELECT ... FOR UPDATE SKIP LOCKED` for task queues.

CHAPTER 08 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- A customer with ₹1,000 can withdraw ₹1,000 simultaneously from two devices (double spending).
- Two cross-transfers (A → B and B → A) can lock each other's accounts, freezing the database into a deadlock.
- ACID properties (specifically Isolation and Atomicity).

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 09

Most Asked & Tricky SQL & DBMS Interview Questions

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Most Asked & Tricky SQL & DBMS Interview Questions

The Senior Engineer Playbook: Concurrency, Three-Valued Logic, Indexing, Window Functions & FinTech Gotchas

 KEY ENGINEERING INSIGHT

Target Role: Senior / Core Developer (3+ Years Experience) | Strategic Projects, Bengaluru

Prerequisites: ANSI SQL, Relational Algebra, Indexing Internals, Transaction Isolation.

Key Focus: 3-Valued Logic bugs, deadlock avoidance, window function edge cases, and high-concurrency ledger design.

 Quick Navigation

1. [Three-Valued Logic \(3VL\) & The Fatal `NOT IN \(NULL\)` Trap](#)
2. [Preventing Double-Spending: Atomic Decrement vs `SELECT ... FOR UPDATE` vs Optimistic Locking](#)
3. [Deadlock Prevention in Two-Account Fund Transfers](#)
4. [Running Totals: `ROWS` vs `RANGE` in Window Frames](#)
5. [`ROW\_NUMBER\(\)` vs `RANK\(\)` vs `DENSE\_RANK\(\)`](#)
6. [Recursive CTEs: Tracing Money Laundering Mule Account Chains](#)
7. [Anti-Joins: `NOT EXISTS` vs `LEFT JOIN ... IS NULL` vs `NOT IN`](#)
8. [Composite B+ Tree Indexing & The Leftmost Prefix Rule](#)
9. [The Gaps and Islands Problem: Consecutive Days of Banking Activity](#)
10. [Aggregate Functions with NULL & The `COALESCE` Imperative](#)

1. Three-Valued Logic (3VL) & The Fatal `NOT IN (NULL)` Trap

 Why Interviewers Ask This

One of the single most dangerous traps in SQL. A query that works perfectly in testing with non-null mock data can return **0 rows** in production the moment a single `NULL` value exists.

🧩 Tricky Query Puzzle

Suppose you have two tables: `accounts` and `frozen_accounts`:

```
-- accounts: 100 active accounts
-- frozen_accounts: IDs [10, 20, NULL]
SELECT *
FROM accounts
WHERE account_id NOT IN (SELECT account_id FROM frozen_accounts);
```

How many rows does this query return?

❌ Intuitive (Wrong) Answer

"98 rows — it excludes accounts 10 and 20, and ignores the NULL."

✅ Actual Shocking Output

0 rows! (Empty Result Set)

🧠 The Mathematical Mechanics: SQL Three-Valued Logic

SQL does not use two-valued boolean logic (`TRUE` / `FALSE`). It uses **Three-Valued Logic**: `TRUE`, `FALSE`, `UNKNOWN`.

When you write `x NOT IN (10, 20, NULL)`, SQL expands it into:

```
(x <> 10) AND (x <> 20) AND (x <> NULL)
```

In SQL, any direct comparison with `NULL` (including `<>` or `=`) evaluates to `UNKNOWN`! Now substitute boolean logic:

```
TRUE AND TRUE AND UNKNOWN ⇒ UNKNOWN
```

A `WHERE` clause only accepts rows where the predicate evaluates to strictly `TRUE`. Because every row evaluates to `UNKNOWN`, **every single account is filtered out!**

Production Fixes

```
-- Fix 1: NOT EXISTS (Best Practice - Immune to NULLs & Fast Semi-Join)
SELECT a.*
FROM accounts a
WHERE NOT EXISTS (
    SELECT 1
    FROM frozen_accounts f
    WHERE f.account_id = a.account_id
);

-- Fix 2: Explicitly filter NULLs inside subquery
SELECT *
FROM accounts
WHERE account_id NOT IN (
    SELECT account_id
    FROM frozen_accounts
    WHERE account_id IS NOT NULL
);
```

2. Preventing Double-Spending: Atomic Decrement vs SELECT ... FOR UPDATE vs Optimistic Locking

Why This Matters at IDFC FIRST Bank

When a customer attempts two simultaneous UPI debit requests (or clicks the payment button twice within 10 milliseconds), both requests might read a balance of ₹1,000 and attempt to deduct ₹700, resulting in a ₹-400 illegal overdraft.

3 Concurrency Strategies Compared

Approach 1: Atomic In-Place Decrement (Fastest & Simplest)

```
UPDATE accounts
SET balance = balance - :debit_amount,
    updated_at = NOW()
WHERE account_id = :acc_id
    AND balance ≥ :debit_amount;
```

- **How it works:** Relies on database row-level locking during the write.
- **Verification in Backend Code:** Inspect the affected rows count (`result.rowCount` in Node.js / `rowcount` in Python).
- If `rowCount == 1`: Deduction succeeded!
- If `rowCount == 0`: Insufficient balance (or account doesn't exist). Return HTTP 400.
- **Trade-off:** No need for multi-statement transactions or explicit lock holding. Ideal for simple single-account debit operations.

Approach 2: Pessimistic Row Locking (`SELECT ... FOR UPDATE`)

```
BEGIN;

-- Acquires an exclusive row-level lock (X Lock) on this specific account
SELECT balance
FROM accounts
WHERE account_id = :acc_id
FOR UPDATE;

-- Application checks: if balance ≥ debit_amount
UPDATE accounts
SET balance = balance - :debit_amount
WHERE account_id = :acc_id;

-- Record double-entry ledger entry
INSERT INTO ledger_entries (account_id, amount, type) VALUES (:acc_id, :debit_amount, 'DEBIT');

COMMIT;
```

- **How it works:** Forces concurrent transactions reading the same row to wait until this transaction commits or rolls back.
- **Trade-off:** Strongest consistency for multi-table banking workflows, but can cause connection pool queuing under massive contention.

Approach 3: Optimistic Locking with Version / Sequence Column

```
-- Step 1: Read balance and current version
SELECT balance, version FROM accounts WHERE account_id = :acc_id;

-- Step 2: Conditional update
UPDATE accounts
SET balance = balance - :debit_amount,
    version = version + 1
WHERE account_id = :acc_id
    AND version = :old_version
    AND balance ≥ :debit_amount;
```

- **Trade-off:** Zero database lock contention, but requires client-side retry loops with exponential backoff if `rowCount = 0`.

3. Deadlock Prevention in Two-Account Fund Transfers

The Deadlock Scenario

- Transaction 1: Transfers money from Account A to Account B.
- Step 1: Locks Account A (`SELECT FOR UPDATE WHERE id = 'A'`)
- Step 2: Attempts to lock Account B...
- Transaction 2 (Simultaneous): Transfers money from Account B to Account A.
- Step 1: Locks Account B (`SELECT FOR UPDATE WHERE id = 'B'`)
- Step 2: Attempts to lock Account A...

- **Result:** Deadlock! Transaction 1 waits for B, while Transaction 2 waits for A. The database engine detects a cycle in the wait-for graph and forcibly aborts one of the transactions.

🛡️ The Definitive Fix: Deterministic Lock Ordering

Always acquire locks in a globally consistent, deterministic order (e.g. sorted by `account_id`):

```
def transfer_funds(from_acc: int, to_acc: int, amount: float):
    # Enforce deterministic order regardless of debit vs credit
    first_id, second_id = sorted([from_acc, to_acc])

    with db.transaction():
        # Lock in strict numerical order:
        db.execute("SELECT 1 FROM accounts WHERE id = :id FOR UPDATE", {"id": first_id})
        db.execute("SELECT 1 FROM accounts WHERE id = :id FOR UPDATE", {"id": second_id})

        # Now execute debit and credit safely with zero deadlock risk:
        db.execute("UPDATE accounts SET balance = balance - :amt WHERE id = :from_id", ...)
        db.execute("UPDATE accounts SET balance = balance + :amt WHERE id = :to_id", ...)
```

4. Running Totals: `ROWS` vs `RANGE` in Window Frames

💡 Why Interviewers Ask This

Window functions are tested in every senior SQL round. Interviewers check if you know why default window framing can cause incorrect running balances when duplicate timestamps exist!

🧩 Tricky Query Puzzle

What is the difference between these two queries?

```
-- Query A:
SUM(amount) OVER (
    PARTITION BY account_id
    ORDER BY txn_date
)

-- Query B:
SUM(amount) OVER (
    PARTITION BY account_id
    ORDER BY txn_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

✅ The Critical Difference

- **Default Frame (Query A):** In ANSI SQL, when `ORDER BY` is present without an explicit frame specification, the default is: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`! `RANGE` treats all rows with **identical `ORDER BY` values (peers)** as a single group! If a customer executes 3 transactions on the exact same date `'2026-03-01'`, `RANGE` will compute the sum of **all three transactions** and show the identical combined total on all three rows!

- **Explicit Physical Frame (Query B):** `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` treats rows **physically line-by-line**, calculating the true step-by-step cumulative running balance as printed on a banking passbook.

```
-- BANKING PASSBOOK STANDARD:
SELECT
    txn_id,
    txn_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY account_id
        ORDER BY txn_date, txn_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_balance
FROM transactions;
```

5. `ROW_NUMBER()` VS `RANK()` VS `DENSE_RANK()`

💡 The Comparison Matrix (Scores: 100, 100, 90, 80)

Value	<code>ROW_NUMBER()</code>	<code>RANK()</code>	<code>DENSE_RANK()</code>	Explanation
100	1	1	1	First tie
100	2	1	1	Tied score gets same rank
90	3	3	2	<code>RANK()</code> skips rank 2; <code>DENSE_RANK()</code> does not skip!
80	4	4	3	Next score

🏠 High-Yield Interview Question: Find the 2nd Highest Account Balance

```
WITH RankedBalances AS (
    SELECT
        account_id,
        balance,
        DENSE_RANK() OVER (ORDER BY balance DESC) AS rnk
    FROM accounts
)
SELECT account_id, balance
FROM RankedBalances
WHERE rnk = 2;
```

Why `DENSE_RANK()` over `LIMIT 1 OFFSET 1`?

Because if two wealthy accounts tie for 1st place with ₹10,00,000, `LIMIT 1 OFFSET 1` will return the second tied account (still ₹10,00,000!). `DENSE_RANK()` correctly returns the true second highest distinct balance tier.

6. Recursive CTEs: Tracing Money Laundering Mule Account Chains

Production FinTech Scenario

A fraud ring launders illicit funds by hopping money through a chain of accounts:

Account 101 → Account 102 → Account 105 → Account 109 .

How do you trace the entire transfer chain from source to final destination using a single SQL query?

Production Recursive CTE Query

```
WITH RECURSIVE TransferChain AS (
  -- Anchor Member: Starting illicit transaction
  SELECT
    from_account,
    to_account,
    amount,
    txn_date,
    1 AS hop_depth,
    ARRAY[from_account, to_account] AS path
  FROM transfers
  WHERE from_account = 101

  UNION ALL

  -- Recursive Member: Find subsequent hops
  SELECT
    t.from_account,
    t.to_account,
    t.amount,
    t.txn_date,
    tc.hop_depth + 1,
    path || t.to_account
  FROM transfers t
  JOIN TransferChain tc ON t.from_account = tc.to_account
  -- Guard against circular infinite money laundering loops:
  WHERE NOT (t.to_account = ANY(tc.path))
    AND tc.hop_depth < 10
)
SELECT hop_depth, from_account, to_account, amount, path
FROM TransferChain
ORDER BY hop_depth;
```

7. Anti-Joins: NOT EXISTS vs LEFT JOIN ... IS NULL vs NOT IN

The Goal

Find all customers who have **never executed a credit card transaction**:

```
-- Method 1: NOT EXISTS (Recommended)
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM card_transactions t
    WHERE t.customer_id = c.customer_id
);

-- Method 2: LEFT JOIN ... WHERE NULL
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN card_transactions t ON c.customer_id = t.customer_id
WHERE t.customer_id IS NULL;
```

Performance & Safety Comparison

1. **NOT EXISTS** : Always safe with **NULL** values. The query planner uses a **Hash Anti-Join** and stops searching as soon as the first matching record is found (Early Exit).
2. **LEFT JOIN ... IS NULL** : Safe with **NULL** s, but requires building the entire joined result set in memory before filtering out rows where the right column is null.
3. **NOT IN** : Extremely dangerous if the subquery returns even a single **NULL** (returns 0 rows).

8. Composite B+ Tree Indexing & The Leftmost Prefix Rule

The Scenario

Suppose you create a composite index on a banking transactions table:

```
CREATE INDEX idx_txn_branch_date_amt ON transactions(branch_code, txn_date, amount);
```

Which of the following queries will effectively use the index?

```
-- Query 1:
SELECT * FROM transactions WHERE branch_code = 'B001' AND txn_date = '2026-03-01';
-- Query 2:
SELECT * FROM transactions WHERE txn_date = '2026-03-01' AND amount > 5000;
-- Query 3:
SELECT * FROM transactions WHERE branch_code = 'B001' AND amount = 5000;
```

Analysis & Leftmost Prefix Rule

- **Query 1: Uses Index!** Matches **branch_code** (leading column) and **txn_date** (second column).
- **Query 2: FAILS to use index (Full Table Scan)!** The leading column **branch_code** is missing. A composite index is sorted like a phonebook (Last Name, First Name). You cannot use a phonebook to look up people by First Name without checking every entry.
- **Query 3: Uses Partial Index.** It uses the index to filter by **branch_code**, but cannot use the index for **amount** because **txn_date** was skipped.

💡 Pro Interview Tip: The Covering Index (**INCLUDE**)

```
CREATE INDEX idx_covering ON transactions(account_id, txn_date) INCLUDE (amount, status);
```

In PostgreSQL, **INCLUDE** stores payload columns directly in the leaf pages of the B+ Tree without adding them to the index search key. This allows **Index-Only Scans** without expensive heap table lookups!

9. The Gaps and Islands Problem: Consecutive Days of Banking Activity

💡 FinTech Scenario

Find customers who have maintained an active transaction streak for **3 or more consecutive days**.

🖥️ Solution using Difference of Dates (**ROW_NUMBER()** Trick)

```
WITH DistinctDays AS (
    -- Step 1: Deduplicate multiple transactions on same day
    SELECT DISTINCT customer_id, CAST(txn_date AS DATE) AS act_date
    FROM transactions
),
NumberedDays AS (
    -- Step 2: Assign a continuous sequence number
    SELECT
        customer_id,
        act_date,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY act_date) AS seq
    FROM DistinctDays
),
Islands AS (
    -- Step 3: (act_date - seq days) remains CONSTANT for consecutive dates!
    SELECT
        customer_id,
        act_date,
        act_date - (seq * INTERVAL '1 day') AS island_group
    FROM NumberedDays
)
SELECT
    customer_id,
    MIN(act_date) AS streak_start,
    MAX(act_date) AS streak_end,
    COUNT(*) AS consecutive_days
FROM Islands
GROUP BY customer_id, island_group
HAVING COUNT(*) ≥ 3
ORDER BY consecutive_days DESC;
```

10. Aggregate Functions with NULL & The **COALESCE** Imperative

💡 Critical Gotchas

1. **COUNT(*)** vs **COUNT(column)** :
2. **COUNT(*)** counts total rows in the partition (including rows where all columns are NULL).

3. `COUNT(column)` counts only rows where `column IS NOT NULL`.
4. `SUM(amount)` on an empty table or all NULLs:
5. Evaluates to `NULL`, NOT `0`!
6. In financial calculations, `NULL + 100` evaluates to `NULL`.
7. **Production Rule:** Always wrap numeric aggregations with `COALESCE`: `sql SELECT COALESCE(SUM(amount), 0) AS total_disbursed FROM loans WHERE status = 'REJECTED';`
8. `AVG(score)` ignores NULLs:
9. If scores are `[10, 20, NULL]`, `AVG` computes `(10 + 20) / 2 = 15`, NOT `30 / 3 = 10`.
10. If you want NULLs treated as zero: `AVG(COALESCE(score, 0))`.

CHAPTER 09 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- How it works: Relies on database row-level locking during the write.
- Verification in Backend Code: Inspect the affected rows count (``result.rowCount`` in Node.js / ``rowCount`` in Python).
- If ``rowCount == 1``: Deduction succeeded!

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 10

SQL & DBMS Last-Minute Interview Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

Banking Critical

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

SQL & DBMS Last-Minute Interview Cheatsheet

 SQL Logical Execution Order

1. FROM & JOIN (Combines tables)
2. ON (Filters join rows)
3. WHERE (Filters base rows before grouping - NO AGGREGATES!)
4. GROUP BY (Aggregates rows into buckets)
5. HAVING (Filters aggregated groups)
6. SELECT (Calculates expressions & column aliases)
7. DISTINCT (Deduplicates output)
8. ORDER BY (Sorts results)
9. LIMIT / OFFSET (Paginates results)

 Window Functions Cheat Sheet

```
-- General Syntax:
FUNCTION() OVER (
  PARTITION BY category_col
  ORDER BY sort_col
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- Key Ranking Differences:
ROW_NUMBER() -- 1, 2, 3, 4 (Always unique sequential integers)
RANK()       -- 1, 2, 2, 4 (Ties get same rank, subsequent rank skipped)
DENSE_RANK() -- 1, 2, 2, 3 (Ties get same rank, NO rank skipped - Top N salaries!)

-- Navigation:
LAG(col, 1)  -- Reads value from previous row (Gap / time difference)
LEAD(col, 1) -- Reads value from next row

-- Running Total (Explicit Frame is faster than default RANGE!):
SUM(amount) OVER (
  PARTITION BY account_id
  ORDER BY created_at
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

ACID & Database Concurrency Cheat Sheet

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Default In
Read Uncommitted	✗ Allowed	✗ Allowed	✗ Allowed	—
Read Committed	✓ Prevented	✗ Allowed	✗ Allowed	Postgres / Oracle
Repeatable Read	✓ Prevented	✓ Prevented	✓ Prevented in Postgres	MySQL (InnoDB)
Serializable	✓ Prevented	✓ Prevented	✓ Prevented	SQLite / High Security

Locking Commands

```

-- Pessimistic Lock (Blocks other readers and writers):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE;

-- Pessimistic Lock (Fail immediately if locked):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE NOWAIT;

-- Task Queue Processing (Skip locked rows to prevent worker pileup):
SELECT * FROM job_queue WHERE status = 'PENDING' LIMIT 1 FOR UPDATE SKIP LOCKED;

-- Optimistic Lock Check:
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE account_id = 101 AND version = :read_version;

```

B+ Tree Indexing Rules

- Leftmost Prefix Rule:** If composite index is on (A, B, C) :
- WHERE A = 1 → **Index Scan (Yes)**
- WHERE A = 1 AND B = 2 → **Index Scan (Yes)**
- WHERE B = 2 AND C = 3 → **Sequential Scan (No index used!)**
- Range Breaks Indexing:** If query has WHERE A = 1 AND B > 5 AND C = 10 :
- Column A uses index seek.
- Column B uses index range scan.
- Column C **cannot** use the index; must be filtered after scan!
- Clustered Index:** Leaf nodes contain full data rows (1 per table).
- Secondary Index:** Leaf nodes contain indexed column + Primary Key pointer.
- Covering Index:** Index contains *all* projected columns, bypassing table data pages entirely.



CHAPTER 10 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- `WHERE A = 1` → Index Scan (Yes)
- `WHERE A = 1 AND B = 2` → Index Scan (Yes)
- `WHERE B = 2 AND C = 3` → Sequential Scan (No index used!)



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases